1. Unidad 3: Acceso a Datos

Tabla de contenidos

1.1. Java Persistence API (JPA) 2
1.2. Spring Data JPA 4
1.2.1. Creación del Proyecto 4
1.2.2. Configuración de Conexión 8
1.2.3. Mapeo de Objeto-Relacional (ORM) 10
1.2.4. Manipulación de Datos 34

En la actualidad la persistencia de datos es parte fundamental en aplicaciones web empresariales, ya que sin ella, no se podría almacenar la información, ni hacer uso de ella posteriormente. Para solventar esa necesidad existen las bases de datos, las cuales permiten almacenar un gran conjunto de datos de forma estructurada para luego extraerlos en el momento que sean necesarios.

En el mercado existen muchos tipos y modelos de bases de datos, pero los más utilizados en las aplicaciones son las bases de datos dinámicas y los modelos de base de datos relacionales (SQL) y no relacionales (NoSQL).

En la siguiente tabla se muestran algunas de las más conocidas:

SQL	**NoSQL**
Oracle	MongoDB
MySQL	ElasticSearch
Microsoft SQL Server	Redis
PostgreSQL	Apache Cassandra
Db2	DynamoDB
SQLite	
MariaDB	

Para dar soporte al conjunto de base de datos SQL y NoSQL listadas en el punto anterior, el equipo de Spring desarrolló un proyecto llamado Spring Data que ofrece un modelo de acceso a los datos basados en la misma estructura consistente del Framework, es decir, que permite la configuración automática, la creación de beans y un gran número de herramientas para persistir y manipular los datos.

Spring Data facilita el uso de tecnologías de acceso a datos, bases de datos relacionales y no relacionales, marcos de reducción de mapas y servicios de datos basados en la nube. Es un proyecto general que contiene muchos subproyectos con funcionalidades específicas para cada tipo de base de datos o tecnología.

A continuación se muestran algunas de sus características principales:

1. **Abstracciones de Repositorio:** Spring Data proporciona una abstracción llamada "Repositorio" que simplifica la creación de operaciones CRUD (Crear, Leer, Actualizar,

Borrar) en una base de datos. Los repositorios permiten definir consultas personalizadas mediante la nomenclatura de métodos y, en muchos casos, generan automáticamente consultas SQL correspondientes.

2. **Soporte para múltiples bases de datos:** Spring Data es compatible con una variedad de bases de datos relacionales (como MySQL, PostgreSQL, Oracle) y no relacionales (como MongoDB, Redis, Cassandra), lo que permite a los desarrolladores cambiar fácilmente entre diferentes tecnologías de almacenamiento de datos sin cambiar su código de aplicación.

3. **Integración con Spring Framework:** Spring Data se integra estrechamente con el ecosistema de Spring Framework. Puede aprovechar otras características de Spring, como la inyección de dependencias, la gestión de transacciones y la seguridad, en sus aplicaciones de persistencia de datos.

4. **JPA (Java Persistence API) y soporte ORM:** Spring Data JPA ofrece soporte para el uso de JPA, lo que facilita la creación de repositorios y consultas relacionales en bases de datos SQL utilizando entidades JPA.

5. **Anotaciones y consultas personalizadas:** Puede definir consultas personalizadas utilizando anotaciones y nombres de métodos en los repositorios de Spring Data. Esto evita la necesidad de escribir consultas SQL en la mayoría de los casos y mejora la seguridad contra ataques de inyección SQL.

6. **Soporte para paginación y ordenamiento:** Spring Data facilita la paginación y el ordenamiento de resultados de consultas, lo que es útil cuando se manejan grandes conjuntos de datos.

7. **Soporte para transacciones declarativas:** Spring Data admite la gestión de transacciones de manera declarativa, lo que permite anotar métodos o clases con anotaciones que definen el comportamiento transaccional sin la necesidad de código adicional.

8. **Soporte para consultas nativas:** Además de las consultas generadas automáticamente, Spring Data permite la ejecución de consultas SQL nativas cuando sea necesario.

9. **Ampliable y personalizable:** Spring Data es altamente configurable y extensible, lo que permite a los desarrolladores personalizar su comportamiento según las necesidades específicas de su aplicación.

En resumen, Spring Data es una poderosa herramienta que simplifica la interacción con bases de datos en aplicaciones basadas en Spring, proporcionando una capa de abstracción y una serie de características que mejoran la productividad del desarrollo y la portabilidad entre diferentes sistemas de gestión de bases de datos.

Al igual que ocurre con Spring Boo y Spring, nosotros sólo haremos uso de uno de los subproyectos de Spring Data, concretamente del subproyecto Spring Data JPA.

1.1. Java Persistence API (JPA)

JPA es la propuesta estándar que ofrece Java para implementar un Framework Object Relational Mapping (ORM), que permite interactuar con la base de datos por medio de objetos, de esta

forma, JPA es el encargado de convertir los objetos Java en instrucciones para el Manejador de Base de Datos (MDB).

Cuando empezamos a trabajar con bases de datos en Java lo primero que se aprende es a utiliza el API de JDBC el cual nos permite realizar consultas directas a la base de datos a través de consultas SQL nativas. JDBC por mucho tiempo fue la única forma de interactuar con las bases de datos, pero representaba un gran problema y es que Java es un lenguaje orientado a objetos y se tenía que convertir los atributos de las clases en una consulta SQL como SELECT, INSERT, UPDATE, DELETE, etc. lo que ocasionaba un gran esfuerzo de trabajo y un provocaba muchos errores en tiempo de ejecución, debido principalmente a que las consultas SQL se tenían que generar frecuentemente al vuelo.

Una de las cosas más importantes para comprender que es JPA es entender que JPA es una especificación y no un Framework como tal, ¿pero quiere decir esto exactamente?, pues bien, una especificación no es más que un documento en el cual se plasman las reglas que debe de cumplir cualquier proveedor que dese desarrollar una implementación de JPA, de tal forma que cualquier persona puede tomar la especificación y desarrollar su propia implementación de JPA, ¿esto quiere decir que pueden existir muchas implementaciones de JPA? la respuesta es sí, de hecho en la actualidad existen varios proveedores como son:

- Hibernate [http://hibernate.org/]
- ObjectDB [http://www.objectdb.com/]
- TopLink [http://www.oracle.com/technetwork/middleware/toplink/overview/index-089172.html]
- EclipseLink [http://www.eclipse.org/eclipselink/]
- OpenJPA [http://openjpa.apache.org/]

Dentro de las implementaciones más utilizadas están Hibernate, EclipseLink & TopLink, las dos primeras son las más utilizadas en el mundo open source y TopLink es muy utilizada en desarrollos y productos relacionados con Oracle. Antes de preguntarnos cuál es la diferencia entre todas estas implementaciones tenemos que comprender que en teoría todas debería de ofrecer la misma funcionalidad y el mismo comportamiento, lo que nos permitiría migrar entre una implementación a otra sin afectar en nada nuestra aplicación.

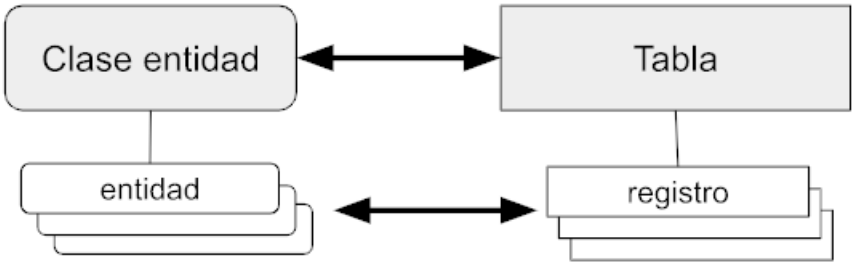
Desde luego esto es solo teoría, ya que en la actualidad no todas las implementaciones implementan al 100% la especificación de JPA, además en escenarios muy concretos puede que se comporten ligeramente diferente, por lo que puede requerir realizar algunos ajustes antes de migrar correctamente de proveedor.

La especificación fue publicada por primera vez en mayo del año 2006 por el proceso de comunidad de Java y poco tiempo después se implementó como parte del lenguaje Java bajo el paquete javax.persistence, otorgando los siguientes beneficios:

- API para dar soporte a las entidades y objetos.
- Implementación de un ORM para el mapeo de objeto relacional.
- Metadatos de los objetos y relaciones existentes en la base de datos.

- Lenguaje de consulta desde Java (JPQL).

JPA nos permite trabajar de forma abstracta y cómoda con las bases de datos relacionales. Provee anotaciones con las que configuramos la equivalencia entre las tablas de la base de datos y ciertas clases especiales. Si las bases de datos relacionales se fundamentan en las tablas, Jakarta Persistence hace lo propio con un concepto equivalente adaptado al mundo de la orientación a objetos: las clases de tipo entidad, que representan a los registros de las tablas.



El ecosistema Spring cuenta con un proyecto específico para la inclusión de las funcionalidades de JPA en cualquier proyecto de forma fácil y rápida, dicho proyecto es Spring Data JPA y será sujeto de estudio en la siguiente sección.

1.2. Spring Data JPA

Spring Data JPA es un módulo de Spring Data para la gestión de acceso a datos que requieren las aplicaciones web de una manera sencilla y basada en las tecnologías y estructuras utilizadas por Spring.

Este módulo crea una capa de abstracción al API de persistencia de Java (JPA), mejorando en gran medida los procesos utilizados para ejecutar consultas, manipular datos, realizar auditoria y reduciendo la cantidad de código repetitivo que se necesita para establecer el acceso a datos en las diferentes tecnologías de base de datos relacionales.

Entre sus principales características podemos destacar:

- Soporte para construir repositorios basados en Spring y JPA.
- Utiliza la librería Querydsl para la creación de sentencias y consultas JPA seguras.
- Soporte de paginación, ejecución dinámica de consultas y capacidad de integrar código de acceso a datos personalizado.

Nota

La implementación por defecto de JPA en Spring Data JPA es Hibernate, aunque puede modificarse en caso de que queramos usar otra implementación distinta.

1.2.1. Creación del Proyecto

A la hora de iniciar un desarrollo basado en Spring Boot lo primero que hay que hacer es crear un Proyecto. Uno de las principales ventajas que presentan los desarrollos con Spring Boot es la autogestión de las dependencias del proyecto, sin embargo, para poder llevar a cabo dicha gestión es indispensable contar con una herramienta que lo facilite.

Spring Boot es totalmente compatible con las dos herramientas más ampliamente utilizadas y elogiadas en la comunidad de desarrolladores, dichas herramientas no son otras que Maven y Gradle.

Nosotros nos decantaremos por Maven, por lo tanto será indispensable contar con la instalación correspondiente para el correcto desarrollo de los distintos proyectos que abordaremos a lo largo del curso.

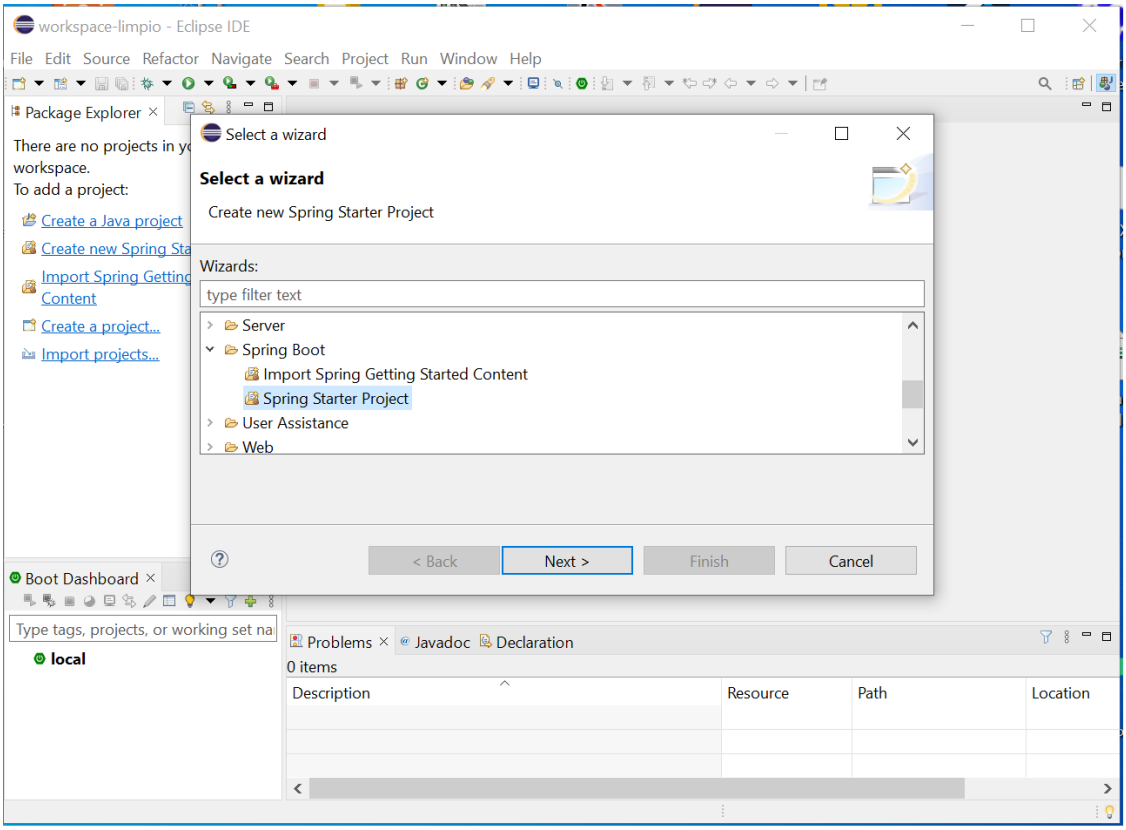
Teniendo en cuenta ésto, vamos a centrarnos en el proceso de creación de un proyecto. Existen varias formas:

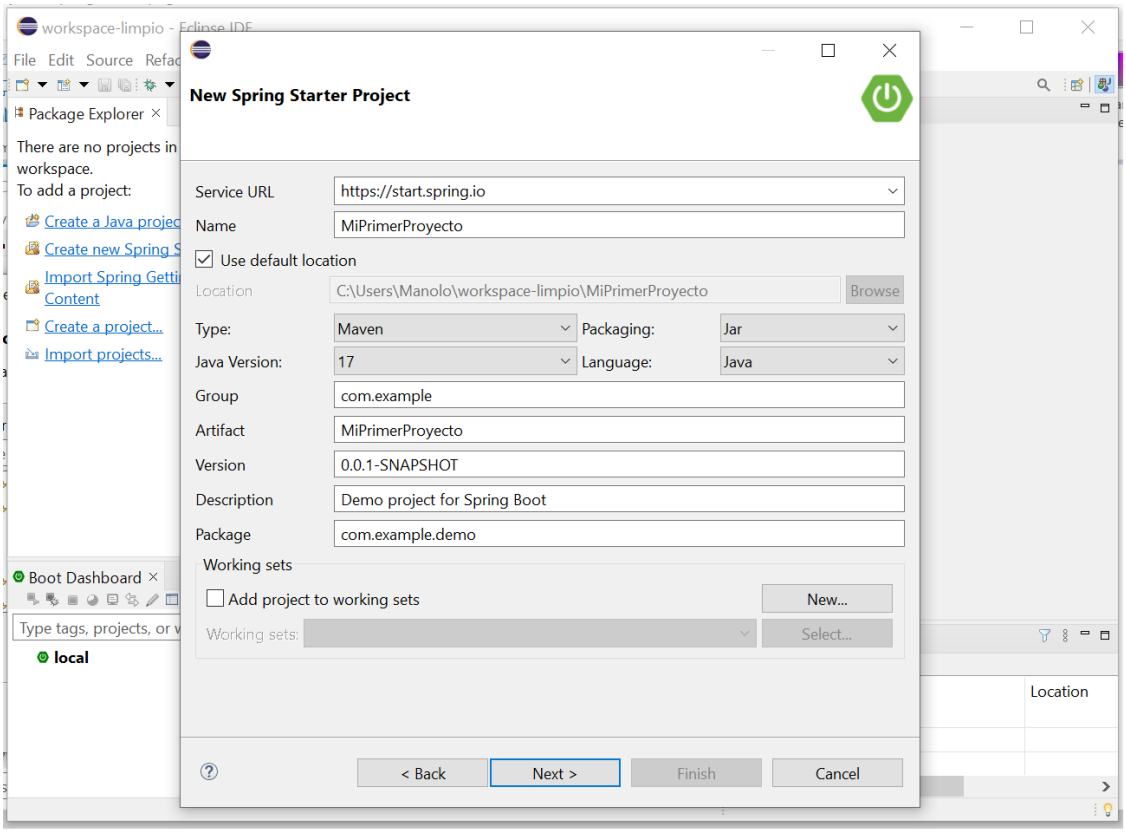
- Instalado *Spring Tools for Eclipse* (que es un Eclipse con Spring Boot integrado). Se puede descargar desde su página oficial: https://spring.io/tools.
- Accediendo a la página https://start.spring.io y generando el proyecto para descargarlo en tu PC. Esta opción sólo nos genera el proyecto, por lo tanto tendremos que contar con IDE adecuado para su posterior edición y manipulación.
- Instalando *Spring Tools* en Eclipse (Se instala desde Eclipse, como un plugin).

En nuestro caso, nos centraremos en la creación del proyecto partiendo de una de las dos últimas opciones (Eclipse + Spring Tools Suite)

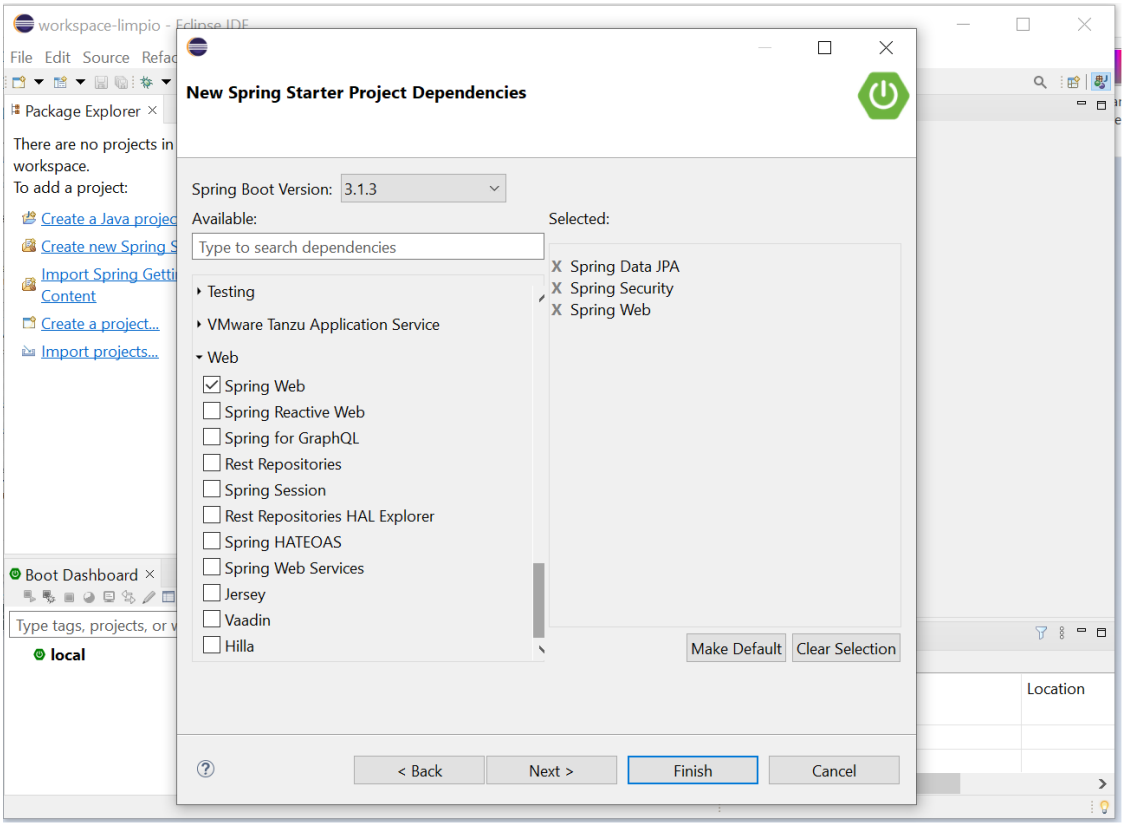
La creación se acomete en dos pasos:

1. Desde eclipse, y partiendo de un workspace limpio, seleccionamos la opción *New/Other/Spring Boot/Spring Starter Project*. Una vez hecho esto, el sistema nos muestra la pantalla de configuración de nuestro proyecto maven:



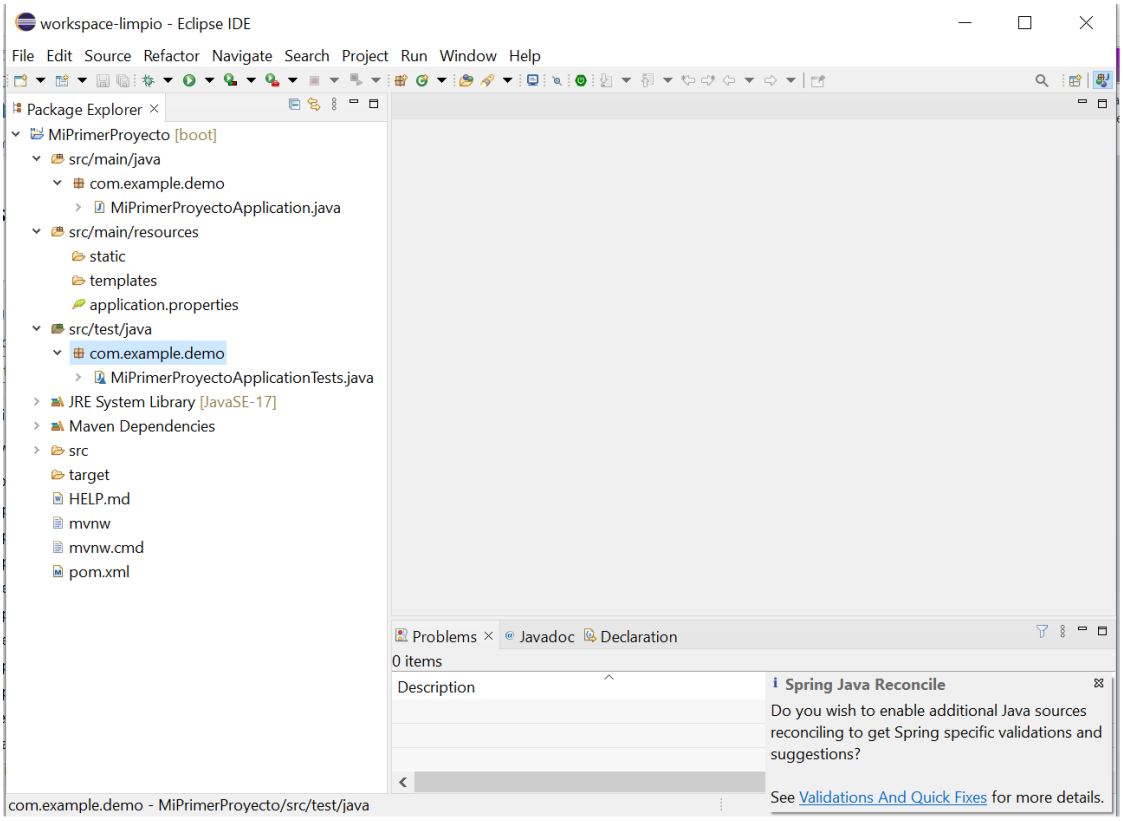


2. Una vez configurados los parámetros principales del proyecto, será necesario especificar las dependencias concretas de nuestro proyecto Spring Boot. Para ellos tendremos que pulsar el botón *Next* y se nos desplegará la pantalla siguiente:



Como podemos observar, desde esta pantalla se puede seleccionar la versión de Spring Boot (en nuestro caso usaremos 3.1.3) así como los subproyectos o dependencias que queremos incluir (para su posterior uso) en el proyecto. Nosotros inicialmente vamos a seleccionar las tres más básicas para el desarrollo de una aplicación web, *Spring Data JPA*, *Spring Security* y *Spring Web*. Una vez configuradas las dependencias sólo nos resta finalizar y automáticamente eclipse nos mostrará el proyecto que acabamos de generar en el explorador de proyectos.

El proyecto resultante presenta la siguiente estructura:



1.2.2. Configuración de Conexión

Una de las principales ventajas de Spring Boot es que te permite conectarte a una base de datos de forma simple y con mínimas configuraciones.

La configuración de la conexión a base de datos se realiza con pocos pasos.

- Define la base de datos que quieres utilizar.
- Incluye las dependencias.
- Define los parámetros de conexión en el archivo de propiedades.

En primer lugar hay que determinar la base de datos que vamos a usar. Como ya hemos comentado, Spring Boot es totalmente compatible con un gran numero de Sistemas de Gestión de Base de Datos, simplemente deberemos incluir las dependencias corresondientes que se limitarán básicamente al conector específico de la base de datos a usar.

En nuestro caso nos decantaremos por **MySQL**, básicamente porque es mucho más ágil para las tareas de aprendizaje y consume muchos menos recursos que otros sistemas como por ejemplo Oracle.

Una vez elegida la base de datos deberemos incluir las dependencias correspondientes, dichas dependencias (como siempre) se gestionan desde el fichero *pom.xml* de nuestro proyecto. A continuación se detalla la dependencia correspondiente para *MySQL* y *Oracle*:

- **MySQL**

```
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.29</version>
</dependency>
```

- **Oracle**

```
<dependency>
    <groupId>com.oracle.database.jdbc</groupId>
    <artifactId>ojdbc8</artifactId>
    <scope>12.2.0.1</scope>
</dependency>
```

Una vez incluidas las dependencias sólo nos resta configurar los parámetros de conexión. Históricamente, los parámetros de conexión a base de datos en las aplicaciones Java se hans establecido a través de código. Sin embargo, con Spring Boot dichos parámetros de conexión, al igual que otros muchos parámetros de configuración, se establecen a través del fichero de configuración **application.properties**. Dicho fichero es uno de los más importantes de toda aplicación Spring Boot, de ahí que lo vayamos a usar en muchas ocasiones a lo largo del curso, su ubicación es la siguiente **Directorio_Proyecto\src\main\resources\application.properties**.

En estas propiedades, primero debemos establecer el path de conexión con la DB y posteriormente indicaremos el user y el password, adicionalmente se especifica el driver y el dialecto. Con esta mínima configuración y la dependencia, ya es suficiente para que Spring Boot pueda establecer conexión con nuestra base de datos.

La configuración mínima para conectar quedaría de la siguiente forma:

Para MySql:

```
spring.datasource.url=jdbc:mysql://localhost:3306/nombre_base_datos
spring.datasource.username=your_user
spring.datasource.password=your_password
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.jpa.database-platform=org.hibernate.dialect.MySQL5InnoDBDialect
```

Para Oracle:

```
spring.datasource.url=jdbc:oracle:thin:@localhost:1521:nombre_base_datos
spring.datasource.username=your_user
spring.datasource.password=your_password
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver
spring.jpa.database-platform=org.hibernate.dialect.Oracle10gDialect
```

Además de estas propiedades, existe otras muchas cuya función puede resultar muy interesante sobre todo en las fases de desarrollo de la aplicación. Un ejemplo es la propiedad ddl-auto.

La propiedada **ddl-auto** permite delegar la gestión de las tablas de la base de datos en Spring, puedes hacerlo agregando simplemente la siguiente propiedad:

```
spring.jpa.hibernate.ddl-auto=create
```

Esta propiedad le indica a Spring que debe crear las tablas desde las entidades que hemos creado.

Además del valor create, la propiedad admite los siguientes valores.

- **none**: no se realizará ninguna acción.
- **drop**: se crearán las tablas a partir de las entidades y al final se borrarán.
- **create**: se borrarán las tablas actuales y al final se crearán de nuevo.
- **update**: realiza una actualización de las tablas del esquema a partir de las entidades.
- **validate**: sólo realiza una validación entre las entidades y las tablas de la base de datos.

Otra opción interesante cuando estamos inmersos en el desarrollo de una aplicación que se conecta con una base de datos es la posibilidad de observar las sentencias SQL que se están ejecutando, para ello podemos utilizar las siguientes propiedades:

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Una vez configurada la conexión, nos adentraremos en lo verdaderamente importante del Spring Data JPA, que no es otra cosa que el proceso de mapeo.

1.2.3. Mapeo de Objeto-Relacional (ORM)

Una de las grandes ventajas de JPA es que nos permite manipular la base de datos a través de objetos, estos objetos son conocidos como Entity, las cuales son clases comunes y corrientes también llamadas POJO's (Plain Old Java Objects). Estas clases tienen la particularidad de ser clases que están mapeadas contra una tabla de la base de datos, dicho mapeo se lleva a cabo generalmente mediante Anotaciones aunque también es posible realizar dicho mapeo mediante ficheros de configuración XML (procedimiento en creciente desuso).

Las anotaciones en Java son fragmentos de código que tienen más funciones que los simples comentarios, pero tienen menos funciones en comparación con las líneas reales de código fuente. En palabras más simples, las anotaciones se utilizan para proporcionar cualquier tipo de información adicional sobre el programa.

Por sí mismas no hacen nada, es al procesarlas cuando se añade su comportamiento, sirve desde para añadir documentación, realizar comprobaciones de compilación, generar código o programar funcionalidades transversales.

Características de las anotaciones en Java

- Todas las anotaciones deben comenzar con '@'.
- Las anotaciones no afectan la acción de un código compilado. Las anotaciones en sí mismas no tienen un impacto directo en la ejecución de un programa Java una vez que ha sido compilado en bytecode.
- Estas anotaciones son una vía para asociar metadatos a los elementos del programa. Los metadatos normalmente comprenden variables de instancia, constructores, métodos, clases, etc.

- Las anotaciones no deben confundirse con los comentarios. Si bien los comentarios son declaraciones no ejecutables del compilador, las anotaciones pueden cambiar la forma en que un compilador ejecuta un programa.

En el caso de Spring Data JPA, dichas anotaciones brindan los suficientes metadatos como para poder por relacionar las clases contra las tablas y las propiedades contra las columnas. Es de esta forma que JPA es capaz de interactuar con la base de datos a través de las clases.

1.2.3.1. Entidades

Las entidades, son POJO's por lo que son simples clases Java como cualquier otra, sin embargo, JPA debe de ser capaz de identificar que clases son Entity para de esta forma poderlas administrar. Es aquí donde nace la importancia de la anotación **@Entity**, esta anotación se debe definir a nivel de clase y sirve únicamente para indicarle a JPA que esa clase es una Entity (Entidad).

```
package com.daw.model;

import jakarta.persistence.Entity;

@Entity
public class Persona {
    ...
    ...
    ...
}
```

Un aspecto importante cuando definimos entidades es si deberemos implementar la interface serializable, dicha interfaz es una interfaz marcadora que se utiliza para indicar que una clase puede ser serializada, es decir, que sus objetos pueden ser convertidos en una secuencia de bytes para que puedan ser guardados en un archivo, transmitidos a través de una red o almacenados en una base de datos y posteriormente deserializados, es decir, convertidos nuevamente en objetos. En general las opiniones están divididas, algunos dicen siempre deberán implementarla y otros dicen que no siempre, la realidad es que la interface serializable sólo es necesaria cuando los objetos son transmitidos fuera de nuestra máquina virtual, como sería el caso del uso de interfaces remotas de EJB, pero por lo general no será necesario implementarla.

Una vez definida la clase como interfaz es necesario identificar la tabla de la base de datos a la que va a referenciar, para ello se hace uso de la anotación **@Table**. La anotación *@Table* es utilizada para indicarle a JPA contra que tabla debe de mapear una entidad, de esta manera cuando se realice una persistencia, borrado o select de la entidad, JPA sabrá contra que tabla de la base de datos deberá interactuar.

La anotación en sí cuenta con las siguientes propiedades;

- **name:** se utiliza para poner el nombre real de la tabla en la base de datos, es recomendable que el nombre sea exacto respetando mayúsculas y minúsculas, sobre todo cuando trabajamos en Linux.

- **schema:** se utiliza para indicar el schema en el que se encuentra la tabla. Esta propiedad por lo general no es necesaria, a menos hagamos uso de varios schemas en nuestra aplicación.

- **indexes:** JPA permite indicar los índex que tiene nuestra tabla, esta opción toma relevancia cuando indicamos que JPA cree las tablas por nosotros (más adelante veremos esta característica).

```
package com.daw.model;

import jakarta.persistence.Entity;

@Entity
@Table(
    name = "persona" ,
    schema = "daw",
    indexes = {@Index(name = "id_index", columnList = "id", unique = true)}
public class Persona {
    ...
    ...
    ...
}
```

Nota

Realmente la anotación @Table no es obligatoria. Si no se proporciona la anotación indicando explícitamente el nombre de la tabla a mapear, JPA utilizará por defecto el nombre de la clase como el nombre de la tabla en la base de datos. Así pues, aún no siendo obligatoria, es una buena práctica utilizarla para ser explícito acerca de la asignación entre la entidad y la tabla, y para permitir una mayor personalización si es necesario.

1.2.3.1.1. Propiedades

Una vez definidas las entidades, pasaremos a mapear cada una de las columnas de la tabla. Cada una de las columnas de la tabla se mapeará usando un atributo o propiedad de la tabla (excepto las columnas de foreing que veremos más adelante).

Llegados a este punto, hay que tener en cuenta que los tipos de datos que manejan los sistemas de gestión de base de datos son muy diversos y no coinciden exactamente con los usados por los lenguajes de programación. Esto no es ningún problema, ya que existe una correspondencia de dichos tipos para permitir la correcta conexión e intercambio de información entre las aplicaciones y las bases de datos.

Dicha correspondencia se muestra en la siguiente tabla:

Java Type	Java DB, Derby, CloudScape	Oracle	DB2	Sybase	MS-SQL Server	MySQL Server
boolean, java.lang.Boolean	SMALLINT	NUMBER(1)	SMALLINT	BIT	BIT	TINYINT(1)
int, java.lang.Integer	INTEGER	NUMBER(10)	INTEGER	INTEGER	INTEGER	INTEGER
long, java.lang.Long	BIGINT	NUMBER(19)	INTEGER	NUMERIC(19)	NUMERIC(19)	BIGINT
float, java.lang.Float	FLOAT	NUMBER(19,4)	FLOAT	FLOAT(16)	FLOAT(16)	FLOAT
double, java.lang.Double	FLOAT	NUMBER(19,4)	FLOAT	FLOAT(32)	FLOAT(32)	DOUBLE
short, java.lang.Short	SMALLINT	NUMBER(5)	SMALLINT	SMALLINT	SMALLINT	SMALLINT
byte, java.lang.Byte	SMALLINT	NUMBER(3)	SMALLINT	SMALLINT	SMALLINT	SMALLINT
java.lang.Number	DECIMAL	NUMBER(38)	DECIMAL(15)	NUMERIC(38)	NUMERIC(28)	DECIMAL(38)
java.math.BigInteger	BIGINT	NUMBER(38)	BIGINT	NUMERIC(38)	NUMERIC(28)	BIGINT
java.math.BigDecimal	DECIMAL	NUMBER(38)	DECIMAL(15)	NUMERIC(38)	NUMERIC(28)	DECIMAL(38)
java.lang.String	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)	VARCHAR(255)
char, java.lang.Character	CHAR(1)	CHAR(1)	CHAR(1)	CHAR(1)	CHAR(1)	CHAR(1)
byte[], java.lang.Byte[], java.sql.Blob	BLOB(64000)	LONG RAW	BLOB(64000)	IMAGE	IMAGE	BLOB(64000)
char[], java.lang.Character[], java.sql.Clob	CLOB(64000)	LONG	CLOB(64000)	TEXT	TEXT	TEXT(64000)
java.sql.Date	DATE	DATE	DATE	DATETIME	DATETIME	DATE
java.sql.Time	TIME	DATE	TIME	DATETIME	DATETIME	TIME
java.sql.Timestamp	TIMESTAMP	DATE	TIMESTAMP	DATETIME	DATETIME	DATETIME

A la hora de hacer el mapeo hay que tener muy en cuenta dichas correspondencias, de lo contrario el mapeo será incorrecto y la aplicación serán incapaz de interactuar con la tabla correspondiente.

Al igual que en las tablas, las entidades también requieren un identificador (*@Id*), dicho identificador deberá de diferenciar a la entidad del resto. El campo ID es importante porque será utilizado por la capa de persistencia a la hora de persistir un objeto, y es por ésto que puede determinar sobre que registro hacer el select, update o delete. JPA soporta ID simples de un solo campo o ID complejos, formados por más de un campo, sin embargo, esta sección abordaremos únicamente los ID simples.

1.2.3.1.1.1. Anotación @Id

Para determinar el ID de una entidad es tan simple como poner la anotación **@Id** sobre la propiedad que sería el ID de la entidad.

El ID puede ser cualquier tipo de datos soportado por JPA, como puede ser, todos los tipos primitivos y clases wraper (*Integer*, *Long*, *Double*, etc), enumeraciones y Calendar. Por norma general es posible utilizar el que más se adapte a tus necesidades, sin embaro, existen un gran número de desarrolladores que recomiendan no usar los tipos primitivos para definir las llaves primarias, sino que recomiendan el uso de las clases wraper. Por ejemplo, en lugar de usar int se recomienda usar Integer, en lugar de usar double usar Double, en lugar de long usar Long, y así para cada tipo de dato.

La razón es muy simple, los tipos primitivos no aceptan valores nulos lo cual hace que no tenga sentido con el principio de la base de datos, por ejemplo, si yo definiera el id de una tabla como long, este tendría como valor inicial 0, aunque en realidad no tenga ID, y en el caso de que fuera de tipo Long (wraper) este iniciaría con valor Null. Esto cuando trabajamos es mucho más manual que validar si una entidad tiene id = 0 o <> 0 para saber que la entidad ya existe en la base de datos.

1.2.3.1.1.1.1. Anotación @GeneratedValue

JPA cuenta con la anotación *@GeneratedValue* para indicarle a JPA que regla de autogeneración de la lleve primaria vamos a utilizar. Por ejemplo, si estamos trabajando con una base de datos MySQL o MS SQL Server el ID debería ser autogenerado sin embargo si trabajamos con Oracle o Postgres el ID debería ser establecido a través de una secuencia. Esto hace necesario tener distintas posibilidades de generación de ID disponibles.

JPA soporta 4 estrategias de autogeneración de la llave primaria:

1.2.3.1.1.1.1.1. Identity

Esta estrategia es la más fácil de utilizar pues solo hay que indicarle la estrategia y listo, no requiere nada más, JPA cuando persista la entidad no enviará este valor, pues asumirá que la columna es auto generada. Esto provoca que el contador de la columna incremente en 1 cada vez que un nuevo objeto es insertado.

```
import java.util.Date;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    ...
    ...
}
```

1.2.3.1.1.1.1.2. Sequence

Mediante esta estrategia le indicamos a JPA que el ID de generar a través de una secuencia de la base de datos. De esta manera, cuando se realice un insert, esta agregara la instrucción para que en el ID se inserte el siguiente valor de la secuencia.

```
import java.util.Date;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
@SequenceGenerator(name="sequenciaDePrueba",
    sequenceName="DB_SEQUENCIA",
    initialValue = 1,
    allocationSize = 1)
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator="sequenciaDePrueba")
    private Long id;
    ...
    ...
}
```

Para poder usar las estrategia Sequence es necesario especificar la secuencia a utilizar mediante la anotación *@SequenceGenerator*.

1.2.3.1.1.1.1.2.1. Anotación @SequenceGenerator

Esta anotación se define por lo general a nivel de clase, y es utiliza para indicarle a JPA que secuencia debe de utilizar para insertar en la base de datos.

La anotación *@SequenceGenerator* tiene las siguientes propiedades:

- **name**: nombre con el que se identifica la secuencia en JPA, este nombre no corresponde al nombre real de la secuencia en la base de datos y debe de coincidir con la propiedad generator de la anotación @GeneratorValue.
- **sequenceName**: nombre real de la secuencia en la base de datos.
- **initialValue**: valor inicial de la secuencia, solo en caso de que JPA la cree.
- **allocationSize**: incremento de la secuencia, solo en caso de que JPA la cree.

1.2.3.1.1.1.1.3. Table

Esta es una estrategia especial de JPA, la cual no depende de la base de datos utilizada como es el caso de Sequence e Identity, puesta esta estrategia crea una tabla únicamente para controlar una secuencia autogenerada por Entidad. Puede resultar confuso, pero en realidad lo que hace esta estrategia es simular la estrategia Identity creando una tabla con dos columnas, la primera el nombre de la entidad y la segunda el último ID utilizado, de esta forma, cuando un nuevo

objeto es insertado consulta el último ID de esa entidad, lo incrementa y lo asigna a la nueva entidad al mismo tiempo que actualiza la tabla con el último ID utilizado.

Esta estrategia es utilizada más que nada por su portabilidad, pero siempre es mejor utilizar las estrategias nativas de la base de datos como secuencias e identity debido a su mejor rendimiento.

Al igual que en la estrategia Sequence, es necesario especificar el generador, en este caso mediante la anotación *@TableGenerator*.

```
import java.util.Date;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
@TableGenerator(
    name = "generadorTableDePrueba",
    initialValue = 1,
    pkColumnName = "ENTITY",
    pkColumnValue = "ID",
    allocationSize = 10,
    table = "ENTITY_GENERATOR"
)
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE, generator="generadorTableDePrueba")
    private Long id;
    ...
    ...
}
```

La anotación *@TableGenerator* tiene las siguientes propiedades:

- **name**: nombre con el que se identifica la tabla en JPA, este nombre no corresponde al nombre real de la tabla en la base de datos y debe de coincidir con la propiedad generator de la anotación *@GeneratorValue*.
- **table**: nombre real de la tabla en la base de datos.
- **pkColumnName**: nombre real de la columna en la que se guardará el nombre de la sequencia.
- **pkColumnValue**: nombre real de la columna en la que se guardara el último ID generado.
- **initialValue**: valor con el que se iniciara la secuencia allocationSize: incremento de la secuencia.

1.2.3.1.1.1.1.4. Auto

Finalmente, esta estrategia lo único que hace es decirle a JPA que utilice la estrategia por default para la base de datos con la que estamos trabajando. Es la menos recomendable, pues dejamos la estrategia a interpretación de JPA y esto nos puedo dejar muchos dolores de cabeza.

1.2.3.1.1.2. Definición de Columnas

Una de las principales características cuando trabajamos con base de datos es que todas las tablas tienen Columnas. Dichas columnas deben mapearse contra los atributos de las entidades, para lo cual es necesario que JPA identifique que columna mapea contra cada atributo de la clase y es aquí donde entra @Column.

1.2.3.1.1.2.1. @Column

La anotación **@Column** nos permitirá definir aspectos muy importantes sobre las columnas de la base de datos de la base de datos como lo es el nombre, la longitud, constrains, etc. En caso de no definir esta anotación en los atributos, JPA determinara el nombre de la columna de forma automática mediante el nombre del atributo, por lo que siempre es recomendable establecer esta anotación en todos los atributos de la clase y evitarnos problemas.

Las propiedades que podemos definirle a *@Column* son las siguientes:

- **name**: Permite establecer el nombre de la columna de la tabla con la que el atributo debe de mapear. A pesar de que ningún atributo de @Column es obligatorio, este atributo siempre recomiendo establecerlo.
- **length**: Permite definir la longitud de la columna en caracteres, solo aplica para Strings, en los demás tipos de datos será omitida. Solo se utiliza cuando JPA creará las tablas.
- **insertable**: Le indica a JPA si esa columna debe ser tomada en cuenta en los inserts, en caso de ser true, el valor será insertado, en caso contrario el valor será omitido y será colocado el valor default de la columna o null.
- **updatable**: Parecido al caso anterior, solo que en este caso la columna se toma en cuenta para operaciones de Update.
- **nullable**: Crea una restricción en la tabla (Not Null) para impedir que se inserten valores nulos. Solo se utiliza cuando JPA creará las tablas.
- **precision**: Determina el número total de dígitos, incluyendo parte entera y decimal si la hubiera. Solo se utiliza cuando JPA creará las tablas.
- **scale**: Utilizada únicamente para columnas que deben tener decimales, como valor recibe el número de decimales. Solo se utiliza cuando JPA creará las tablas.
- **table**: Esta propiedad es muy avanzada y la analizaremos más adelante. Se utiliza cuando existe herencia en las entidades y las columnas pueden estar distribuidas en más de una tabla.
- **unique**: Creara una restricción en la tabla para que sea el valor de esa columna sea única. Solo se utiliza cuando JPA creará las tablas.

Para comprender un poco mejor como funciona la anotación @Column vamos a ver un breve ejemplo:

```
import java.util.Date;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;
```

```
    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;

    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    ...
    ...
    ...

}
```

Vemos que el atributo id, solo requiere definir la propiedad *name*, pues por default, ya es única, no nula, no actualizable. Por lo que con definir el nombre será necesario.

En el caso del nombre, definimos la propiedad *name*, le indicamos que no puede ser nulo y le estamos definiendo una longitud de 150 caracteres.

En el caso del salario también ponemos el nombre, le indicamos que el suledo puede tener 2 decimales y que no debe de ser nulo.

Finalmente, en la fecha de registro también establecemos el nombre, pero en este caso veamos que el nombre de la columna es diferente que el de la propiedad y es cuando es especialmente necesario definir la propiedad name, le indicamos que no debe de ser nulo y finalmente, la fecha de ingreso no debe de ser actualizable.

A partir de la anotación *@Column* y las propiedades que hemos visto anteriormente es posible mapear la práctica totalidad de las columnas de cualquier base de datos, sin embargo, existe un gran número de anotaciones adicionales que permiten/facilitan la manipulación de datos mucho más específicos y que no vamos a desarrollar en este curso. Dentro de estas anotaciones podemos destacar *@Temporal*, *@Enumerated* o *@Transient*.

1.2.3.1.1.2.2. Objets Pesados @Lob

JPA nos permite mediante la anotación *@Lob* mapear objetos pesados de base de datos. Podemos considerar como objetos pesados imágenes, xml, binarios, cadenas de texto extensas, json o cualquier objeto que pueda tener un tamaño muy grande o de longitud indefinida.

La anotación **@Lob** es lo único que se requiere para indicarle a JPA que ese campo es un objeto pesado y que debe de tratarse como tal. Por lo general se utiliza los arreglos de bytes (*byte []*), ya que permite almacenar cualquier cosa.

La anotación *@Lob* no tiene ningún atributo, por lo que solo será necesario definirla para que funcione. Otro punto importante es que esta anotación creará una columna de tipo longblob en MySQL y podría variar según el manejador de base de datos utilizado, pero al final siempre será un campo para objetos pesados.

A continuación podemos ver un ejemplo de uso:

```
import java.util.Date;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
```

```
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;

    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    @Column(name = "FOTO" ,nullable = true)
    @Lob()
    private byte[] foto;

    ...
    ...
    ...

}
```

1.2.3.1.1.2.3. Estrategia de carga con @Basic

Habitualmente la anotación *@Lob* se usa conjuntamente con la anotación *@Basic*. *@Basic* es una anotación que nos permite controlar el momento en que una propiedad es cargada desde la base de datos, evitando traer valores que no son necesarios al momento de cargar el objeto. Esta anotación es utilizada generalmente para anotar objetos pesados, como una imagen o un archivo binario.

En JPA existen dos conceptos que son claves para entender cómo JPA carga los objetos desde la base de datos y que son vitales para mejorar el rendimiento de la aplicación, estos conceptos se explican a continuación:

- **Lazy loading (Carga demorada)**: Los objetos de carga demorada no serán cargados desde la base de datos cuando el objeto sea creado, pero será cargado en cuanto se acceda a la propiedad. De esta manera JPA identifica cuando la propiedad es accedida por primera vez para cargar el valor desde la base de datos.
    - **@Basic(fetch = FetchType.LAZY)**
- **Eager loading (Carga ansiosa o temprana)**: Este es la utilizada por default para la mayoría de las propiedades en JPA, a excepción de las colecciones las cuales las analizaremos mas adelante.
    - **@Basic(fetch = FetchType.EAGER)**

La anotación *@Basic* define además la propiedad **optional**, la cual es de tipo booleano, si se le indica true, se decimos a JPA que esta propiedad es opcional y en ella puede haber un valor Null, en caso de ser false, le indicamos que la propiedad siempre deberá tener valor. Esta propiedad es más que nada para ayudar a JPA a optimizar los query para obtener los valores, pero tendrá que corresponderse con la realidad, ya que si marcamos la propiedad como no opcional (obligatorio) y finalmente la propiedad tiene valor null en la base de datos JPA lanzará un error.

NOTA: optional es diferente de nullable de @Column, pues nullable sirve para definir la estructura de la columna y optional son sugerencias a JPA para optimizar los querys. Nunca debemos de confundir eso.

Un punto importante es que *@Basic* se puede emplear prácticamente para cualquier valor como tipos *primitivos*, clases *wrapper*, *Calendar, enums, Time, Date, Timestamp. byte[], char[]*, clases que implementan *java.lang.Serializable*, etc.

Vamos a ver como se usa a partir del ejemplo anterior:

```
import java.util.Date;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;

    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    @Column(name = "FOTO" ,nullable = true)
    @Basic(optional = false, fetch = FetchType.EAGER)
    @Lob()
    private byte[] foto;

    ...
    ...
    ...

}
```

1.2.3.1.2. Relaciones

Una vez descrito cómo se mapean las propiedades/columnas con JPA vamos a analizar el procedimiento de mapeo de las relaciones mediante JPA. Vamos a analizar las diferentes cardinalidades soportadas por JPA, y estudiaremos cómo una entidad puede relacionarse con otra y cómo son afectadas las entidades al estar relacionadas.

1.2.3.1.2.1. Relaciones OneToOne

Las relaciones One to One se caracterizan porque sólo puede existir una y sólo una relación con la Entidad de destino. Para mapear este tipo de relaciones se utiliza la anotación **@OneToOne**.

La entidad marcada como **@OnoToOne** deberá tener una referencia a la entidad destino y por ningún motivo podrá ser una colección. De la misma forma, la entidad destino no podrá pertenecer a otra instancia de la entidad origen.

Solo para ponernos en contexto, las relaciones *@OneToOne* se utilizan cuando existe una profunda relación entre la entidad origen y destino, de tal forma que la entidad destino le

pertenece a la entidad origen y solo a ella, por ende, la existencia de la entidad destino depende de la entidad origen.

La anotación *@OneToOne* cuenta con los siguientes atributos:

- **cascade**: Las operaciones que se deben conectar en cascada al objetivo de la asociación (Opcional).
- **fetch**: Si la asociación debe cargarse de forma diferida o debe recuperarse con entusiasmo (Opcional).
- **mappedBy**: El campo propietario de la relación (Opcional).
- **optional**: Si la asociación es opcional (Opcional).
- **targetEntity**: La clase de entidad que es el objetivo de la asociación (Opcional).

Para comprender mejor su funcionamiento vamos a usar un ejemplo basado en una entidad *Persona* y una entidad *Operador*. Pueden existir personas sin operador asociado pero no puede existir ningún operador sin estar asociado a una persona. Veamos como se modela este caso usando una relación OneToOne.

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;

    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    @Column(name = "FOTO" ,nullable = true)
    @Basic(optional = false, fetch = FetchType.EAGER)
    @Lob()
    private byte[] foto;

    @OneToOne(optional = true, fetch = FetchType.EAGER)
    @JoinColumn(name = "OPERADOR", referencedColumnName = "ID", nullable = true)
    private Operador operador;

}
```

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;

@Entity
@Table( name = "operador")
public class Operador {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column (name="USUARIO")
    private String usuario;

    @Column (name="CLAVE")
    private String clave;

    @OneToOne(mappedBy = "operador")
    private Persona persona;

    ...
    ...

}
```

Algo que hay que tener en cuenta, es que tanto la entidad *Persona* como *Operador*, tienen anotada como *@OneToOne* a la otra entidad, sin embargo, sólo *Persona* tiene una columna para hacer el JOIN, esto se debe a que ésta es la que tiene la anotación *@JoinColumn*, la cual nos permite establecer como se llamará la columna para realizar el JOIN.

Adicionalmente, la entidad Operador tiene definida la propiedad *mappedBy* de la anotación *@OneToOne* a través de la cual se especifica que la entidad *Persona* tiene una propiedad que representa la relación y que permite la relación bidireccional.

1.2.3.1.2.1.1. @JoinColumn

La anotación **@JoinColumn** se puede llegar a confundir con *@Column*, sin embargo, tiene una funcionalidad diferente. *@JoinColumn* se utiliza para marcar una propiedad que requiere de un JOIN para poder ser accedida, mientas que *@Column* se utiliza para representar columnas simples que no están relacionadas con otra Entidad. A pesar de que se utiliza para cosas distintas, la realidad es que se parecen muchísimo, pues tiene casi las mismas propiedades, por que podríamos decir que *@JoinColumn* es el equivalente de *@Column* cuando utilizamos relaciones con Entidades.

Las propiedades de la anotación *@JoinColumn* son las siguientes:

- **name**: Indica el nombre con el que se deberá de crear la columna dentro de la tabla.
- **referencedColumnName**: Se utiliza para indicar sobre que columna se realizará el Join de la otra tabla. Por lo general no se suele utilizar, pues JPA asume que la columna es el ID de la Entidad objetivo.
- **unique**: Crea una constraints en la tabla para impedir valores duplicados (default false).
- **nullable**: Crea una constraints en la tabla para impedir valores nulos (default true).
- **insertable**: Le indica a JPA si este valor deberá guardarse en la operación de inserción (default true).
- **updatable**: Le indica a JPA si el valor deberá actualizarse durante el proceso de actualización (default true).
- **columnDefinition**: Esta propiedad es utiliza para indicar la instrucción SQL que se deberá utilizar para crear la columna en la base de datos. Esta nos ayuda a definir exactamente como se creará la columna sin depender de la configuración de JPA.

- **table**: Le indicamos sobre que tabla deberá realizar el JOIN, normalmente no es utilizada, pues JPA asume la tabla por medio de la entidad objetivo.

- **foreignKey**: Le indica a JPA si debe de crear el Foreign Key, esta propiedad recibe uno de los siguientes valores CONSTRAINT , NO_CONSTRAINT , PROVIDER_DEFAULT definidos en la enumeración *javax.persistence.ForeignKey*.

La anotación @JoinColumn es indispensable tanto en las relaciones OneToOne como en las de tipo ManyToOne.

1.2.3.1.2.2. Relaciones ManyToOne

Se considera que existe una relación muchos a uno (**@ManyToOne**) entre dos tablas cuando muchos registros de la primera tabla se pueden corresponder con un único registro de la segunda tabla. A nivel de JPA, ésto se traduce en que va a existir una entidad (muchos) que tendrá una columna que hará referencia a otra entidad (uno), mientras que en ésta última no existirá ninguna referencia a la otra entidad (unidireccional) o existirá una propiedad que será de tipo List, Set, … a través de la cual se podrán recuperar las entidades relacionadas (birireccional).

La anotación *@ManyToOne* cuenta con los siguientes atributos:

- **optional**: indica si la relación es opcional, es decir, si el objeto puede ser null. Esta propiedad se utiliza optimizar las consultas. Si JPA sabe que una relación es opcional, entonces puede realizar un RIGHT JOIN o realizar la consulta por separado, mientras que, si no es opcional, puede realizar un INNER JOIN para realizar una solo consulta.

- **cascade**: Esta propiedad le indica que operaciones en cascada puede realizar con la Entidad relacionada, los valores posibles son ALL , PERSIST , MERGE , REMOVE , REFRESH , DETACH y están definidos en la enumeración *javax.persistence.CascadeType* (ésto lo desarrollaremos con más detalle en otro apartado posterior).

- **fetch**: Esta propiedad se utiliza para determinar cómo debe ser cargada la entidad, los valores están definidos en la *enumeración javax.persistence.FetchType* y los valores posibles son:

  - **EAGER** (ansioso): Indica que la relación debe de ser cargada al momento de cargar la entidad.

  - **LAZY** (perezoso): Indica que la relación solo se cargará cuando la propiedad sea leída por primera vez.

- **targetEntity**: Esta propiedad recibe una clase (Class) la cual corresponde a la clase de la relación. No suele ser utilizada, pues JPA puede inferir la clase por el tipo de la propiedad.

Vamos a intentar comprender su funcionamiento usando el siguiente ejemplo:

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;

    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    @Column(name = "FOTO" ,nullable = true)
    @Basic(optional = false, fetch = FetchType.EAGER)
    @Lob()
    private byte[] foto;

    @OneToOne(optional = true, fetch = FetchType.EAGER)
    @JoinColumn(name = "OPERADOR", referencedColumnName = "ID", nullable = true)
    private Operador operador;

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    @JoinColumn(name = "CIUDAD", referencedColumnName = "ID", nullable = false)
    private Ciudad ciudad;

}
```

Ahora tendremos dos opciones:

1. **Relación Unidireccional**: en este caso la entidad *Ciudad* no tiene refencia alguna a la entidad *Persona* y por lo tanto no podrá recuperar esa información a partir de un objeto *Ciudad*.

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;

@Entity
@Table (name="CIUDAD")
public class Ciudad {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column (name="CODIGO")
    private String codigo;

    @Column (name="DESCRIPCION")
    private String descripcion;

    ...
    ...

}
```

2. **Relación Bidireccional**: en este caso la entidad *Ciudad* debe tener una referencia a la entidad *Persona*, y además será de tipo colección o listado para poder recuperar todas las ciudades relacionadas. Para mapear esta situación usaremos la anotación *@OneToMany* que veremos en el siguiente apartado.

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;

@Entity
@Table (name="CIUDAD")
public class Ciudad {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column (name="CODIGO")
    private String codigo;

    @Column (name="DESCRIPCION")
    private String descripcion;

    @OneToMany(mappedBy = "ciudad")
    private Set<Persona> personas;

    ...
    ...

}
```

En nuestro caso, hemos definido la propiedad **optional** en false, pues toda Persona debe de tener forzosamente una Ciudad. La propiedad **fetch** la definimos en *EAGER* , aun que para este tipo de relación es su valor por default y no sería necesario definirlo.

1.2.3.1.2.3. Relaciones OneToMany

Las relaciones uno a muchos (**@OneToMany**) se caracterizan por Entidad donde tenemos un objeto principal y colección de objetos de otra Entidad relacionados directamente. Estas relaciones se definen mediante colecciones, pues tendremos una serie de objetos pertenecientes al objeto principal.

La anotación *@OneToMany* cuenta con los siguientes atributos:

- **cascade**: Las operaciones que se deben conectar en cascada al objetivo de la asociación (Opcional).
- **fetch**: Si la asociación debe cargarse de forma diferida o debe recuperarse con entusiasmo (Opcional).
- **mappedBy**: El campo propietario de la relación (Opcional).
- **targetEntity**: La clase de entidad que es el objetivo de la asociación (Opcional).

Para entender el funcionamiento de la anotación *@OneToMany* vamos a usar un ejemplo una vez más:

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;
```

```
    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    @Column(name = "FOTO" ,nullable = true)
    @Basic(optional = false, fetch = FetchType.EAGER)
    @Lob()
    private byte[] foto;

    @OneToOne(optional = true, fetch = FetchType.EAGER)
    @JoinColumn(name = "OPERADOR", referencedColumnName = "ID", nullable = true)
    private Operador operador;

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    @JoinColumn(name = "CIUDAD", referencedColumnName = "ID")
    private Ciudad ciudad;

    @OneToMany(mappedBy = "persona")
    private Set<Direccion> direcciones;

}
```

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table (name="DIRECCION")
public class Direccion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column (name="DESCRIPCION")
    private String descripcion;

    @ManyToOne
    @JoinColumn(name = "PERSONA", nullable = false)
    private Persona persona;

    ...
    ...

}
```

El funcionamiento es idéntico a la relación ManyToOne bidireccional que hemos visto en el apartado anterior.

1.2.3.1.2.4. Relaciones ManyToMany

Las relaciones Mucho a Muchos (**@ManyToMany**) se caracterízan por Entidades que están relacionadas con muchos elementos de un tipo determinado, pero al mismo tiempo, estos últimos registros no son exclusivos de un registro en particular, sino que pueden ser parte de varios, por lo tanto, tenemos una Entidad A, la cual puede estar relacionada como muchos registros de la Entidad B, pero al mismo tiempo, la Entidad B puede pertenecer a varias instancias de la Entidad A.

Algo muy importante a tomar en cuenta cuando trabajamos con relaciones *@ManyToMany*, es que en realidad este tipo de relaciones no existen físicamente en la base de datos, y en su lugar, es necesario crear una tabla intermedia que relaciones las dos Entidades, veremos más adelante como resolvemos eso.

Consideremos el siguiente ejemplo:

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;

@Entity
@Table(name = "persona")
public class Persona {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE", nullable = false, length = 150)
    private String nombre;

    @Column(name = "SUELDO", nullable = false, scale = 2)
    private double sueldo;

    @Column(name = "FECHA_REGISTRO", updatable = false, nullable = false)
    @Temporal(TemporalType.DATE)
    private Calendar fechaRegistro;

    @Column(name = "FOTO" ,nullable = true)
    @Basic(optional = false, fetch = FetchType.EAGER)
    @Lob()
    private byte[] foto;

    @OneToOne(optional = true, fetch = FetchType.EAGER)
    @JoinColumn(name = "OPERADOR", referencedColumnName = "ID", nullable = true)
    private Operador operador;

    @ManyToOne(optional = false, fetch = FetchType.EAGER)
    @JoinColumn(name = "CIUDAD", referencedColumnName = "ID")
    private Ciudad ciudad;

    @OneToMany(mappedBy = "persona")
    private Set<Direccion> direcciones;

    @JoinTable(name = "REL_PERSONA_HOBBY", joinColumns = @JoinColumn(name = "PERSONA", nullable = false),
 inverseJoinColumns = @JoinColumn(name = "HOBBY", nullable = false))
    @ManyToMany(cascade = CascadeType.ALL)
    private Set<Hobby> hobbies;

}
```

```
import java.util.Set;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.ManyToMany;
import jakarta.persistence.Table;

@Entity
@Table (name="HOBBY")
public class Hobby {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column (name="DESCRIPCION")
    private String descripcion;

    @ManyToMany(mappedBy = "hobbies")
    private Set<Persona> personas;

    ...
    ...
}
```

Como podemos apreciar, hemos creado una lista de tipo Hobby, la cual es anotada con *@ManyToMany*. Adicionalmente, hemos definido la anotación **@JoinTable**, la cual nos sirve

para definir la estructura de la tabla intermedia que contendrá la relación entre los libros y los autores.

La anotación **@JoinTable** no es obligatoria en sí, ya que en caso de no definirse JPA asumirá el nombre de la tabla, columnas, longitud, etc. Para no quedar a merced de la implementación de JPA, siempre es recomendable definirla, así, tenemos el control total sobre ella.

Hemos definidos las siguientes propiedades de la anotación *@JoinTable*:

- **name**: Nombre de la tabla que será creada físicamente en la base de datos.
- **joinColumns**: Corresponde al nombre para el ID de la Entidad Book.
- **inverseJoinColumns**: Corresponde al nombre para el ID de la Entidad Author

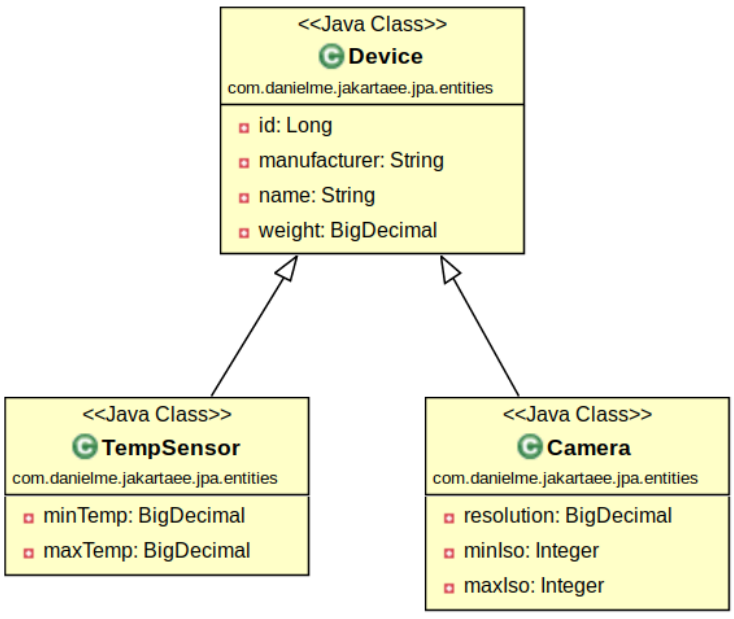
El caso de la entidad *Hobby* es más simple, pues solo marcamos la colección con *@ManyToMany*, pero en este caso ya no es necesario definir la anotación *@JoinTable*, en su lugar, definimos la propiedad *mappedBy* para indicar la relación bidireccional y al mismo tiempo, JPA puede tomar la configuración del *@JoinTable* de *Persona*.

1.2.3.1.3. Herencia

La herencia es una de las características definitorias de la programación orientada a objetos, aunque en la actualidad se le tenga cierta aversión debido a su empleo abusivo, es una técnica fundamental en la definición de modelos de datos y en la aplicación de algunos patrones de diseño.

Dado que el modelo relacional no contempla el mecanismo de herencia, JPA define cuatro estrategias fáciles de utilizar para vincular jerarquías de entidades con tablas. Ninguna de ellas se puede considerar como solución «universal» y la elección se basará en nuestras necesidades, valorando en todo momento los beneficios e inconvenientes de cada opción.

Para desarrollar la explicación nos basaremos en el siguiente modelo de ejemplo.



Hay dos tipos de dispositivos: cámara y sensor de temperatura. No está de más recordar que las entidades pueden heredar de clases que no lo sean y viceversa, así como ser abstractas.

Sea cual sea la estrategia elegida, siempre se configura en la clase raíz de toda la jerarquía, que en nuestro ejemplo es *Device*.

1.2.3.1.3.1. Ignorar la jerarquía (mapped superclass)

Lo más fácil es no trasladar la jerarquía a la base de datos y declarar como entidades solo las «hojas» del árbol. En el ejemplo, tendríamos dos entidades, *TempSensor* y *Camera*, con sus respectivas tablas. En ambas encontraremos los atributos de la clase padre *Device*.

```
@Entity
@Table(name = "cameras")
public class Camera extends Device {

    private BigDecimal resolution;

    private Integer minIso;

    private Integer maxIso;

}
```

```
@Entity
@Table(name = "temp_sensors")
public class TempSensor extends Device {

    private BigDecimal minTemp;

    private BigDecimal maxTemp;

}
```

Se trata de dos entidades «normales». No tienen identificador porque es uno de los atributos que heredan de *Device*.

```
@MappedSuperclass
public class Device {

    @Id
    private Long id;

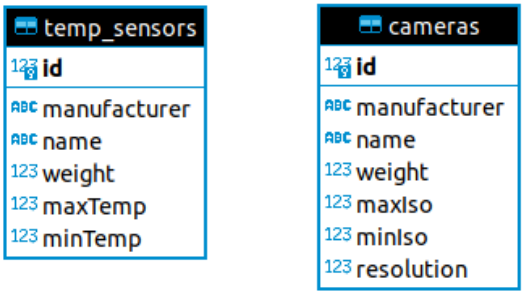
    private String manufacturer;

    private String name;

    private BigDecimal weight;
}
```

Device se ha marcado con @MappedSuperclass [https://jakarta.ee/specifications/persistence/3.0/apidocs/jakarta.persistence/jakarta/persistence/mappedsuperclass] para que Hibernate aplique a las clases hijas las configuraciones de persistencia que contiene. En el ejemplo, se trata del identificador.

La base de datos queda como sigue.



El inconveniente de esta configuración es, precisamente, que *Device* no es una entidad. No podremos usarla en consultas para obtener todos los dispositivos de forma genérica, ni tampoco en relaciones. Por ejemplo, una entidad llamada *Almacen* que contenga un listado con los dispositivos que alberga, independientemente de su tipo. Las jerarquías en las que la superclase deba ser una entidad con relaciones son muy habituales.

1.2.3.1.3.2. Una tabla para todo (single table)

La estrategia Single Table es la estrategia predeterminada y consiste en guardar todas las entidades en una única tabla.

```
@Entity
@Table(name = "devices")
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "DTYPE")
@DiscriminatorValue("Device")
public class Device {

    @Id
    private Long id;

    private String manufacturer;

    private String name;

    private BigDecimal weight;

}
```

```
@Entity
@DiscriminatorValue("TempSensor")
public class TempSensor extends Device {

    private BigDecimal minTemp;

    private BigDecimal maxTemp;

}
```

```
@Entity
@DiscriminatorValue("Camera")
public class Camera extends Device {

    private BigDecimal resolution;

    private Integer minIso;

    private Integer maxIso;

}
```

Tendremos una tabla *devices* que almacena todos los registros.

```
DTYPE     |id|manufacturer|name          |weight|maxIso|minIso|resolution|maxTemp|minTemp|
----------|--|------------|--------------|------|------|------|----------|-------|-------|
Camera    | 1|            |camera test   |      |  3000|    50|     50.00|       |       |
TempSensor| 2|            |sensor test   |      |      |      |          |  90.00| -20.00|
Device    | 3|            |Generic device|      |      |      |          |       |       |
```

La columna *DTYPE* permite saber la entidad a la que corresponde el registro. Si el nombre *DTYPE* no nos parece adecuado, es posible personalizarlo con la anotación *DiscriminatorColumn*. El nombre identificativo de cada entidad es personalizable gracias a la propiedad *DiscriminatorValue*.

Se trata de una estrategia muy fácil y eficiente. Es la estrategia que ofrece el mejor rendimiento: las operaciones CRUD sobre las entidades son directas y no requieren *join*. Lo que no es tan magnífica es la definición de la tabla debido a que tendremos numerosos valores nulos. Para las cámaras, las columnas propias de los sensores de temperatura no tendrán valor y viceversa. Además, las columnas que no pertenezcan a la clase padre no pueden marcarse como no nulas porque las entidades que no las tuvieran no se podrían insertar, lo que comprometerá la integridad de los datos dentro del modelo relacional. Esto último se puede corregir implementando *triggers* en la propia base de datos.

Otro problema de esta tabla única es que tiende a ser muy grande tanto a lo ancho (muchas columnas) como a lo largo (muchos registros). Lo que provoca que con el tiempo pueda llegar a ser enorme.

1.2.3.1.3.3. Una tabla por clase (table per class)

Esta es la traslación directa de las entidades a la base de datos, ignorando que mantienen una relación de herencia. Cada entidad no abstracta tiene su propia tabla con todos sus atributos, incluyendo los heredados. No hay claves ajenas que relacionen estas tablas.

```
@Entity
@Table(name = "devices")
@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
public class Device {

    @Id
    private Long id;

    private String manufacturer;

    private String name;

    private BigDecimal weight;

}
```

```
@Entity
@Table(name = "temp_sensors")
public class TempSensor extends Device {

    private BigDecimal minTemp;

    private BigDecimal maxTemp;

}
```

```
@Entity
@Table(name = "cameras")
public class Camera extends Device {
```

```
    private BigDecimal resolution;

    private Integer minIso;

    private Integer maxIso;

}
```

La estructura de base de datos para esta estrategia sería la siguiente:



Se trata de una estratégia muy simple y eficiente siempre y cuando obtengamos las entidades situadas en las hojas del árbol de la jerarquía. Un *Device* es un *Device* (obvio), pero las cámaras y sensores también lo son. Esto obliga a Hibernate a realizar una unión entre todas las tablas para averiguar y obtener el dispositivo deseado (puede ser una cámara o un sensor).

Así pues, esta estrategia no es la más aconsejable si vamos a trabajar intensamente con las entidades de la parte superior de la jerarquía porque penaliza el rendimiento de su explotación.

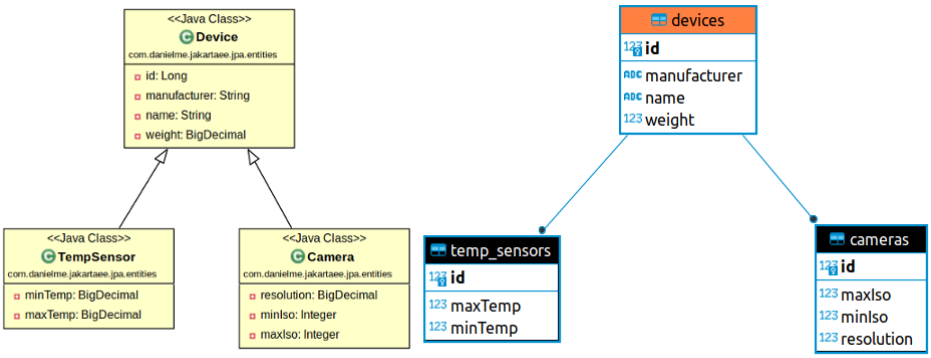
Es importante destacar que las claves primarias deben ser únicas para todas las tablas de la jerarquía, de lo contrario no podremos trabajar con las entidades que no sean «hojas». Este requerimiento nos obliga a descartar el uso de identificadores de tipo identidad (IDENTITY) porque son únicos solo para cada tabla y pueden repetirse entre varias de ellas. JPA asegura que esta configuración no se produzca porque cuando la detecte lanzará esta excepción.

Si con TABLE_PER_CLASS queremos que los identificadores sean generados por la base de datos (muy recomendable), lo mejor es recurrir a una secuencia (SEQUENCE) para obtener los identificadores de todos los registros de las tablas correspondientes a la jerarquía. La configuración resulta sencilla, pues, tal y como hemos visto, el identificador solo se declara en la entidad raíz. Eso sí, en MySQL no tenemos está opción, pues no cuenta con secuencias.

1.2.3.1.3.4. Tablas unidas (joined table)

La cuarta y última estrategia crea una tabla por entidad, incluso para las abstractas, con los atributos que define cada una sin incluir los heredados. Cada tabla está relacionada con su tabla \entidad padre con una clave ajena, también usada como clave primaria, para poder recuperar todos los datos de una entidad (los suyos y los heredados) reuniendo las tablas . La estructura que tenemos en la base de datos es la más parecida a una jerarquía de clases que el modelo relacional permite.

En la siguiente imagen, las vemos frente a frente.



La configuración, como siempre, se realiza definiendo el tipo de herencia en la clase raíz.

```
@Entity
@Table(name = "devices")
@Inheritance(strategy=InheritanceType.JOINED)
public class Device {

    @Id
    private Long id;

    private String manufacturer;

    private String name;

    private BigDecimal weight;

}
```

```
@Entity
@Table(name = "temp_sensors")
@PrimaryKeyJoinColumn(name="id")
public class TempSensor extends Device {

    private BigDecimal minTemp;

    private BigDecimal maxTemp;

}
```

```
@Entity
@Table(name = "cameras")
@PrimaryKeyJoinColumn(name="id")
public class Camera extends Device {

    private BigDecimal resolution;

    private Integer minIso;

    private Integer maxIso;

}
```

Como podemos observar, en cada clase hija debemos indicar el nombre del campo que contiene la clave ajena (la FK) para hacer el *join* con el padre de la jerarquía.

Esta traslación tan natural de las entidades a tablas no está exenta de inconvenientes. Por ejemplo, para guardar una nueva cámara es necesario realizar la inserción de dos registros para una sola entidad (uno en *devices* y otro en *cameras*).

A su vez, la obtención de la cámara requiere el *join* entre *cameras* y *devices* porque sus datos están repartidos en las dos tablas. Por tanto, cuantas más clases conformen la jerarquía, más operaciones serán necesarias para explotarla. Si es grande y con muchos niveles, será recomendable analizar cuidadosamente si nos conviene seguir otra estrategia.

1.2.3.1.3.5. Conclusión

Como hemos visto, no hay ninguna solución que sea óptima para todas las situaciones, por lo tanto, según nuestras condiciones tendremos que escoger la estrategia que más se ajuste a nuestros intereses.

Podríamos destacar la estrategia «SINGLE_TABLE» como la más óptima en cuanto a rendimiento, y la estrategia «JOINED» como la más flexible. En cuanto a al estrategia «TABLE_PER_CLASS» habrá que tener especial cuidado al elegirla por sus problemas de rendimiento trabajando con polimorfismo y por que nos podemos encontrar con proveedores de JPA que no la implementen adecuadamente o directamente no la implementen.

1.2.4. Manipulación de Datos

Spring Data JPA se fundamenta en el concepto de **repositorio**. Un repositorio no es más que una clase que se encarga de gestionar todas las operaciones de persistencia contra la base de datos. La creación de repositorios genéricos reducen de forma significativa el código que tenemos que construir facilitando y agilizando el desarrollo de las aplicaciones.

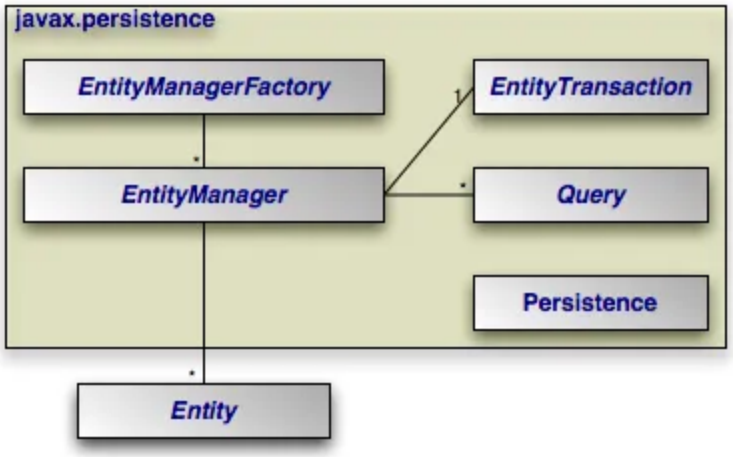
Los repositorios JPA son clases específicas marcadas con la anotación @Repository, dicha anotación dota a nuestra clase de todas las funcionalidades necesarias para la interacción con la base de datos.

Las mayoría de los repositorios implementados en Spring Data JPA extenderán del repositorio genérico y añadirán más operaciones. Sin embargo, antes de centrarnos en los repositorios JPA vamos a centrarnos en un concepto básico para el correcto funcionamiento de la persistencia y su adecuada implementación.

1.2.4.1. EntityManager

JPA tiene como interface medular al EntityManager, el cual es el componente que se encarga de controlar el ciclo de vida de todas las entidades definidas en la unidad de persistencia, y es mediante esta interface que se pueden realizar las operaciones básicas de una base de datos, como consultar, actualizar, borrar, crear (CRUD). También es la clase por medio de la cual se controlan las transacciones.

La arquitectura de persistencia implementada en JPA sigue el siguiente esquema:



Las entidades viven en un contexto de persistencia (PersistenceContext) (es decir, almacén de datos) que define el alcance en el que ocurre el ciclo de vida de la entidad, como la creación, actualización, eliminación, etc. En otras palabras, el contexto de persistencia es básicamente una caché para administrar la entidad e interactuar con la base de datos cuando sea necesario y actualizar/modificar/eliminar la entidad. Para gestionar el ciclo de vida de las entidades, se utiliza la instancia **EntityManager**.

La instancia de EntityManager proporciona métodos mediante los cuales podemos interactuar con el contexto de persistencia.

Los contextos de persistencia donde viven las entidades y el administrador de entidades son de dos tipos: **administrados por contenedores** y **administrados por aplicaciones**.

El contexto persistente administrado por contenedor garantizará que la instancia de EntityManager esté disponible cuando se use dentro de los componentes empresariales como @Component @Service @Controller, etc. Además, se asegurará de que se utilice cualquier transacción que exista entre los componentes y la instancia de EntityManager y luego el contexto se propague entre los componentes. ***Debemos pensar en el alcance de la transacción cuando realizamos la llamada de un componente de servicio a otro o creamos un controlador para el componente de servicio.*** En el **contexto de persistencia administrado por contenedores**, tenemos la flexibilidad de inyectar administradores de entidades y utilizar métodos en ellos. Precisamente por todas estas carácterísticas **esta modalidad es la más utilizada en las aplicaciones web basadas en Spring Boot**, por este motivo será la modalidad que nosotros estudiaremos en el presente curso.

Las operaciones de interacción con la base de datos en Spring Boot se realizan desde los denominados Repositorios.

1.2.4.2. Repositorio Básico

A continuación vamos a desgranar un ejemplo básico de Repositorio.

```
@Repository
public class TesterRepository {

 @PersistenceContext
 private EntityManager em;

 public List<Persona> findAll() {

  TypedQuery<Persona> query = em.createQuery("SELECT p FROM Persona p", Persona.class);
  return query.getResultList();

 }

}
```

Como se puede observar, nuestro repositorio está anotado **@Repository** (esto nos permitirá inyectarlo sin problema en otras clases de nuestra aplicación).

El primer elemento a destacar en el repositorio es la inyección del EntityManager.

```
@PersistenceContext
private EntityManager em;
```

A través de esta inyección, ya tendremos acceso al EntityManager asociado a nuestro contexto de persistencia y podremos hacer uso de el para interactuar con a base de datos.

Posteriormente, tenemos el único método de nuestro repositorio, que en nuestro caso es una búsqueda sin condiciones sobre la entidad *Persona*. Dicho mètodo no recibe parámetros de entrada y devuelve el listado de personas generado por la consulta.

Vamos a desgranar dicho contenido. En primer lugar se debe generar la query a ejecutar, dicha consulta se implementa usando el lenguaje de consulta de JPA conocido como JPQL.

```
TypedQuery<Persona> query = em.createQuery("SELECT p FROM Persona p", Persona.class);
```

La query se crea a partir del método *createQuery* proporcionado por el EntityManagger, una vez generada la query se lanza su ejecución.

```
return query.getResultList();
```

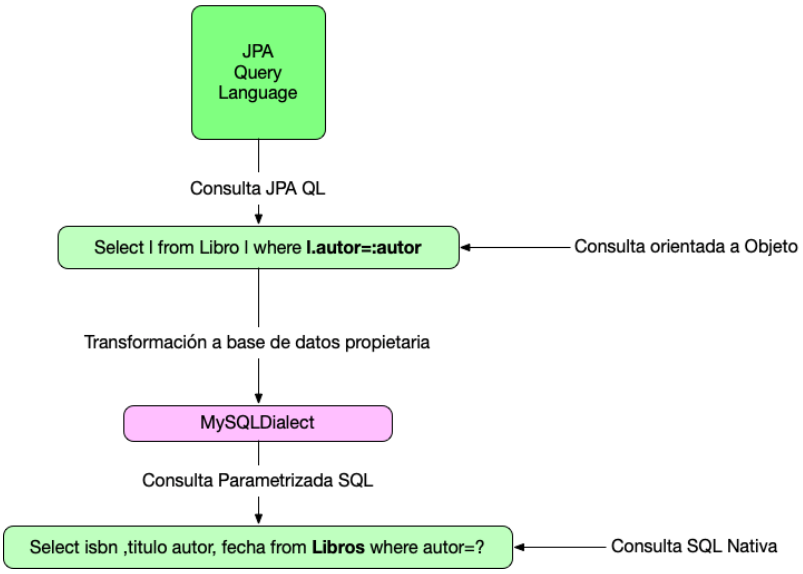
El listado de resultados no será más que un List de entidades Persona que prodrán ser procesados en la clase que invoque dicho método.

Existe una gran variedad de mecanismos para la ejecución de consultas en JPA, la mayoría de ellas apoyadas en JPQL, de ahí que en la siguiente sección la dediquemos exclusivamente a introducirnos a dicho lenguaje.

1.2.4.3. Introducción a JPQL

El Java Persistence Query Language (JPQL) es el lenguaje estándar de consultas de JPA. Es un lenguaje diseñado para combinar la simplicidad de la semántica y sintaxis del lenguaje SQL con la expresividad de un lenguaje orientado a objetos.

JPQL no es SQL. A pesar de las similitudes en los dos lenguajes en términos de estructura y palabras clave hay diferencias importantes. Las similitudes en los lenguajes son intencionadas ya que permite a los desarrolladores con conocimientos de SQL que la curva de aprendizaje sea menor, sin embargo, la naturaleza orientada a objetos del lenguaje JPQL requiere un manera de pensar diferente.



JPQL es un lenguaje de consulta de entidades/objetos en lugar de tablas y registros. El lenguaje nos proporciona una manera de expresar consultas en términos de entidades y sus relaciones operando sobre las entidades en lugar de sobre el modelo de BBDD.

JPQL permite abstraernos de la BBDD, es decir, las consultas son escritas sobre el modelo de entidades/objetos sin necesidad de conocer como estas son mapeadas a la BBDD.

1.2.4.3.1. Sentencias SELECT

Son las sentencias más significativas, y permiten la recuperación de un conjunto de entidades de la BBDD. Su estructura es la siguiente:

```
SELECT e
FROM Employee e
```

El dominio definido en la cláusula *FROM* no es una tabla sino una entidad.

1.2.4.3.2. Cláusula SELECT

La cláusula *SELECT* no enumera los campos o utiliza un "*" para seleccionar todos los campos. En su lugar, utiliza la variable "e" que indica que el tipo de resultado es una entidad *Employee*

```
SELECT e.department
FROM Employee e
```

La cláusula *SELECT* utiliza una expresión de ruta para entidad *Deparment*.

```
SELECT e.name, e.salary
FROM Employee e
```

La cláusula *SELECT* utiliza múltiples expresiones que determinan que el tipo de resultado es un *Object* con dos elementos, el primero un *String* asociado al name y un *Long* asociado al salary.

1.2.4.3.3. Cláusula FROM

Se usa para declarar una o más variables de identificación. Las variables de identificación son el punto de entrada para las expresiones de consultas. Debe existir al menos una variable de identificación definida en la cláusula *FROM*.

Cuando una declaración de variable de identificación es únicamente un nombre de Entidad se conoce como declaración de variable de rango.

JOIN

Un Join es una consulta que combina resultados de distintas entidades. Cuando la consulta es traducida a SQL es común que las join entre entidades produzcan join similares entre las tablas.

Inner Join

Un inner join devuelve todas las entidades del lado izquierdo que tienen una relación con la entidad del lado derecho de la relación

Se pueden definir de forma explícita utilizando el operador *JOIN* en la cláusula *FROM*.

```
SELECT p
FROM Employee e JOIN e.phones p
```

Su traducción a SQL será:

```
SELECT p.id, p.phone_num, p.type, p.emp_id
```

```
FROM emp e, phone p
WHERE e.id = p.emp_id
```

Otra manera es mediante una variable de identificación en la cláusula *FROM* y condiciones de unión sobre ella en la cláusula *WHERE*. Este tipo de consulta se utiliza cuando no hay una relación explícita entre dos entidades en el modelo de dominio.

```
SELECT DISTINCT d
FROM Department d, Employee e
WHERE d = e.department
```

Outer Join

Un outer join devuelve todas las entidades del lado izquierdo de la relación y la entidad de lado derecho con la que se relaciona.

```
SELECT e, d
FROM Employee e LEFT JOIN e.department d
```

Su traducción a SQL será:

```
SELECT e.id, e.name, e.salary, e.manager_id, e.dept_id, e.address_id,
d.id, d.name
FROM employee e LEFT OUTER JOIN department d
ON (d.id = e.department_id)
```

En el ejemplo anterior sino existe departamento el segundo objeto del Objeto devuelto será null.

Fetch Join

Permite seleccionar una o varias relaciones que deben ser traídas de forma “ansiosa” (EAGER), es decir, modificar el comportamiento por defecto que lo que hace es cargar las entidades relacionadas de forma “perezosa” (LAZY)

```
SELECT e
FROM Employee e JOIN FETCH e.address
```

1.2.4.3.4. Cláusula WHERE

Se utiliza para añadir condiciones de filtrado y reducir el número de resultados de la consulta. Veremos a continuación ejemplos de las expresiones condicionales más importantes.

BETWEEN permite filtrar por un rango de valores inicial y final (incluidos) de tipo

Numeric, *String* o *Date*.

```
SELECT e
FROM Employee e
WHERE e.salary BETWEEN 40000 AND 45000
```

LIKE permite filtrar por un patrón. El carácter “%” representa cualquier carácter o caracteres.

```
SELECT d
FROM Department d
WHERE d.name LIKE '__Eng%'
```

IN permite filtrar que un valor único está dentro de una lista de valores. En este caso el estado debe ser NY o CA.

```
SELECT e
FROM Employee e
WHERE e.address.state IN ('NY', 'CA')
```

IS EMPTY o *IS NOT EMPTY* filtra por colecciones vacías o viceversa

```
SELECT e
FROM Employee e
WHERE e.directs IS NOT EMPTY
```

EXISTS o *NOT EXISTS* permite filtra por subconsultas que devuelven algún valor

```
SELECT e
FROM Employee e
WHERE NOT EXISTS (SELECT p
FROM e.phones p
WHERE p.type = 'Cell')
```

ANY, *ALL* permite filtrar por subconsultas para indicar que alguna entidad debe cumplir la condición o todas deben cumplirla

```
SELECT e
FROM Employee e
WHERE e.directs IS NOT EMPTY AND
e.salary < ALL (SELECT d.salary
FROM e.directs d)
```

1.2.4.3.5. Cláusula ORDER

Permiten especificar el orden de los resultados. Las palabras clave ASC y DESC son opcionales y colocadas a continuación de la expresión de ordenamiento indican el sentido de la ordenación (ascendente o descendente). Por defecto sino se indica nada el orden es ascendente

```
SELECT e, d
FROM Employee e JOIN e.department d
ORDER BY d.name, e.name DESC
```

1.2.4.3.6. Paso de Parámetros

Una de los aspectos a la hora de implementar consultas JPQL soon los parámetros de consulta. Los parámetros de consulta son una forma de construir y ejecutar consultas parametizadas.

Así que, en lugar de implentar la consulta de la siguiente forma:

```
SELECT * FROM employees e WHERE e.emp_number = '123';
```

Lo haremos usando consultas parametrizadas:

```
SELECT * FROM employees e WHERE e.emp_number = ?;
```

Donde el símbolo *?* será sustituido por el valor de un parámetro que será indicado a la hora de ejecutar la consulta.

Al igual que los parámetros de la declaración preparados por JDBC, JPA especifica dos maneras diferentes de escribir consultas parametizadas:

- Parámetros de posición.

- Parámetros nombrados.

Nota

Podemos usar parámetros posicionales o nombrados, pero no debemos mezclarlos dentro de la misma consulta.

1.2.4.3.7. Parámetros de Posición.

El uso de parámetros posicionales se basa en la asignación de parámetros en base a la posición de los mismos en la consulta. Veamos un ejemplo:

```
TypedQuery<Employee> query = em.createQuery("SELECT e FROM Employee e WHERE e.empNumber = ?1",
 Employee.class);
String empNumber = "A123";
Employee employee = query.setParameter(1, empNumber).getSingleResult();
```

Como se puede observar, declaramos estos parámetros dentro de la consulta escribiendo un signo de interrogación, seguido de un número entero positivo (*?1*). Se inicia con *1* y se avanza, aumentando por uno cada vez.

Podemos utilizar el mismo parámetro más de una vez dentro de la misma consulta, lo que hace que estos parámetros sean más similares a los parámetros nombrados.

La numeración de parámetros es una característica muy útil, ya que mejora la usabilidad, la legibilidad y el mantenimiento. Vale la pena mencionar que las consultas nativas de SQL también soportan la unión de parámetros posicionales.

Los parámetros también pueden ser una lista o colección de valores:

```
TypedQuery<Employee> query = entityManager.createQuery(
  "SELECT e FROM Employee e WHERE e.empNumber IN (?1)" , Employee.class);
List<String> empNumbers = Arrays.asList("A123", "A124");
List<Employee> employees = query.setParameter(1, empNumbers).getResultList();
```

1.2.4.3.8. Parámetros Nombrados

Los parámetros nombrados son bastante similares a los parámetros posicionales; sin embargo, permite elaborar consultas más legibles ya que cada parámetro tendrá un nombre específico:

```
TypedQuery<Employee> query = em.createQuery(
  "SELECT e FROM Employee e WHERE e.empNumber = :number" , Employee.class);
StringempNumber = "A123";
Employee employee = query.setParameter("number", empNumber).getSingleResult();
```

Podemos ver que declaramos el parámetro con dos puntos (*:*), seguido de un identificador de cadena (identificador JPQL), que es un marcador de posición para el valor real que configuramos en tiempo de ejecución. Antes de ejecutar la consulta, tenemos que establecer el parámetro o los parámetros mediante lanzando el método *setParameter*.

Una cosa interesante a señalar es que ***TypedQuery* soporta la cadena de métodos,** que se vuelve muy útil cuando hay que establecer múltiples parámetros.

```
TypedQuery<Employee> query = em.createQuery(
  "SELECT e FROM Employee e WHERE e.name = :name AND e.age = :empAge" , Employee.class);
String empName = "John Doe";
int empAge=55;
```

```
List<Employee> employees = query
  .setParameter("name", empName)
  .setParameter("empAge", empAge)
  .getResultList();
```

Aquí estamos recuperando a todos los empleados con un nombre y una edad de un dado. Como vemos claramente, y uno puede esperar, podemos construir consultas con **múltiples parámetros y tantas ocurreciones de ellos como sea necesario.**

Al igual que ocurre con los parámetros de posición, los parámtros nombrados también soportan colecciones o listas:

```
TypedQuery<Employee> query = entityManager.createQuery(
  "SELECT e FROM Employee e WHERE e.empNumber IN (:numbers)" , Employee.class);
List<String> empNumbers = Arrays.asList("A123", "A124");
List<Employee> employees = query.setParameter("numbers", empNumbers).getResultList();
```

1.2.4.4. Elaboración de Consultas (Query vs TypedQuery)

JPA posee las interfaces ***Query*** y ***TypedQuery*** para configurar y ejecutar consultas. La interfaz Query se utiliza para los casos en los que los resultados se esperan que sean de tipo Objeto (hay que hacer un *casting* en caso de necesitar devolver un tipo concreto).

La interfaz TypeQuery extiende a Query, por lo que una consulta con tipo puede ser tratada como si fuera una consulta sin tipo, pero no al revés. Se obtiene una implementación de la interfaz apropiada para una consulta dada a través de uno de los métodos factoría de la interfaz del EntityManager. La elección de un método u otro depende del tipo de la consulta.

Una consulta se define dinámicamente pasando la cadena de la consulta JPQL y el tipo de resultado esperado al método *createQuery()* de la interfaz *EntityManager*. El tipo de resultado puede ser omitido para obtener consultas sin tipo. No existen restricciones a la hora de definir consultas.

La habilidad de construir una cadena en tiempo de ejecución y utilizarla para una definición de la consulta es útil, especialmente para aplicaciones donde el usuario puede especificar criterios complejos y la forma exacta de la consulta no puede ser conocida con anticipación. Como se señaló anteriormente, además de consultas de cadena dinámica, JPA también es compatible con una API de criterios para crear consultas dinámicas con objetos Java.

Una cuestión a considerar con cadenas de consultas dinámicas, sin embargo, es el costo de la traducción de JPQL en SQL para su ejecución. Un motor de consulta típica tendrá que analizar la cadena de JPQL en un árbol de sintaxis, obtener los metadatos de mapeo objeto-relacional para cada entidad en cada expresión, y luego generar el SQL equivalente. Para las aplicaciones de muchas consultas, el costo de procesamiento de la consulta dinámica puede ser un problema importante de rendimiento.

1.2.4.4.1. Consultas con Nombre (NamedQuery)

Las consultas con nombre son una poderosa herramienta para la organización de las definiciones de consulta y la mejora de rendimiento de la aplicación. Una consulta con nombre se define mediante la anotación *@NamedQuery*, que puede ser puesta en la definición de clase para cualquier entidad. La anotación define el nombre de la consulta, así como el texto de la consulta.

```
@NamedQuery(name="findSalaryForNameAndDepartment",
```

```
            query="SELECT e.salary " +
                  "FROM Employee e " +
                  "WHERE e.department.name = :deptName AND " +
                  " e.name = :empName")
```

El nombre de la consulta es del ámbito de toda la unidad de persistencia por lo que debe ser único dentro de ese ámbito de aplicación. Ésta es una restricción importante a tener en cuenta, como comúnmente se utilizan nombres de consulta como "FindAll" tendrá que ser calificado para cada entidad. Una práctica común es el prefijo en el nombre de la consulta con el nombre de la entidad. Por ejemplo, el "findAll" de consulta para los empleados de la entidad se denominará "Employee.findAll".

Debido a que la cadena de consulta se define en la anotación, no pueden ser alterados por la aplicación en tiempo de ejecución. Esto contribuye al rendimiento de la aplicación y ayuda a prevenir el tipo de problemas de seguridad que discutimos. Debido a la naturaleza estática de la cadena de consulta, cualquier criterio adicional que se requiere para la consulta debe ser especificado utilizando parámetros de consulta.

En este caso la ejecución se lanzaría de la siguiente forma:

```
TypedQuery<Integer> consulta = em.createNamedQuery("findSalaryForNameAndDepartment", Integer.class);
consultaAlumnos.setParameter("deptName", "Desarrollo").setParameter("empName", "Manuel").getResultList();
```

1.2.4.4.2. Consultas Nativas (NativeQuery)

Una *NativeQuery* es simplemente una consulta SQL ejecutada desde JPA. Estas consulas nos permiten liberar toda la potencia de nuestra base de datos, ya que podemos utilizar funciones patentadas (específicas) no disponibles en la sintaxis restringida de JPQL.

Aunque tiene sus ventajas, las queries nativas también tienen sus inconvenientes. El princiapl es que perdemos la portabilidad de nuestra aplicación con *NativeQuery* porque nuestro proveedor de JPA ya no puede abstraer detalles específicos de la implementación o proveedor de la base de datos.

Veamos un ejemplo:

```
Query nativeQuery = em.createNativeQuery("SELECT * FROM users WHERE id=:userId", UserEntity.class);
nativeQuery.setParameter("userId", id).getSingleResult();
```

La mayoría de las veces, una buena consulta JPQL puede satisfacer nuestras necesidades y, lo más importante, mantener un nivel de abstracción de la implementación de la base de datos real.

Utilizar *NativeQuery* no significa necesariamente casarnos con un proveedor de bases de datos específico. Después de todo, si nuestras consultas no utilizan comandos SQL patentados y están utilizando sólo una sintaxis SQL estándar, los proveedores de conmutación no deben ser un problema.

1.2.4.5. Operaciones de Creación (Persist)

El concepto de JPA Persist es uno de los conceptos más elementales de JPA pero a veces cuesta entender a detalle como funciona a la hora de persistir datos contra la base de datos.

Para desarrollar esta sección vamos a apollarnos en un ejemplo sencillo. Consideremos las siguiente entidad:

```
@Entity
@Table (name="CIUDAD")
public class Ciudad {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column (name="CODIGO")
    private String codigo;

    @Column (name="DESCRIPCION")
    private String descripcion;

    ...

}
```

Teniendo en cuenta esta entidad, para poder insertar una nueva ciudad en la base de datos necesitaremos un método de creación en nuestro repositorio:

```
public Ciudad create(String codigo, String descripcion) {
    Ciudad nueva_ciudad = new Ciudad();
    nueva_ciudad.setCodigo(codigo);
    nueva_ciudad.setDescripcion(descripcion);
    em.persist(nueva_ciudad);
    return nueva_ciudad;
}
```

Como se observa, en primer lugar debemos de crear la nueva entidad, bien usando el consturctor vacío y seteando posteriormente los atributos o bien usando un constructor específico con los parámetros correspondientes. Una vez creada la entidad debemos persistir la entidad en base de datos, para lo cual usamos el método *persist*.

Nota

Para que se pueda persistir la entidad es indispensable contar con una transacción activa, la forma de garantizar dicha transacción es usar la anotación *@Transactional* en nuestro método de persistencia.

1.2.4.6. Operaciones de Actualización/Modificación

La actualización/modificación de una entidad es muy similar a la creación, con la diferencia de que la modificación debe de realizarse sobre una entidad previamente recuperada de base de datos.

Veamos el siguiente ejemplo:

```
public Ciudad update(Long id, String codigo, String descripcion) {
    TypedQuery<Ciudad> query = em.createQuery("SELECT c FROM Ciudad c WHERE c.id = :id" , Ciudad.class);
    Ciudad ciudad = query.setParameter("id", id).getSingleResult();
    ciudad.setCodigo(codigo);
    ciudad.setDescripcion(descripcion);
    em.merge(ciudad);
    return ciudad;
}
```

Como podemos observar, en primer lugar debemos de recuperar la entidad a modificar a partir de su *id*. Para ello usamos una consulta a la que se le pasa el id como parámetro.

Una vez recuperada, modificamos sus atributos usando los métodos *set*, para posteriormente guardar los cambios usando el método *merge*.

Aunque en el ejemplo la entidad a actualizar se recupera previamente de base de datos, existe la posibilidad de lanzar el merge sobre una entidad no gestionada por el *EntityManager*. En dicho caso es necesario especificar el identificador de la entidad, y será el propio *merge* el que gestione la recuperación de los datos necesarios y lance la actualización.

```
public Ciudad update(Long id, String codigo, String descripcion) {
    Ciudad ciudad = new Ciudad();
    ciudad.setId(id);
    ciudad.setCodigo(codigo);
    ciudad.setDescripcion(descripcion);
    em.merge(ciudad);
    return ciudad;
}
```

Una de las principales diferencias entre los dos métodos, es que en el segundo de los casos, es indispensable inicializar todos los atributos de la entidad. De lo contrario, la actualización los pondrá a null.

Las acualizaciones también pueden realizarse usando el método *persist* , siempre y cuando tengamos claro lo que estamos haciendo. Las funcionalidades de **persist** y **merge** son similares aunque tienen importantes diferencias.

1.2.4.6.1. Diferencia entre persist y merge

entityManager.persist(objeto)

1. Inserta el objeto Java como un nuevo registro en la base de datos.
2. Si el objeto ya existe, persiste los cambios en base de datos.
3. Añade el objeto Java al Entity Manager y pasa a ser administrado por la misma.

entityManager.merge(objeto)

1. Busca un objeto Java ya administrado por el Entity Manager con la misma identificación.
2. Si existe, actualiza en la base de datos y devuelve el objeto ya administrado.
3. Si no existe, inserta el nuevo registro en la base de datos y pasa a ser administrado.

1.2.4.7. Operaciones de Borrado

El caso del borrado también sigue la misma estructura. Veamos el ejemplo:

```
public void delete(Long id) {
    TypedQuery<Ciudad> query = em.createQuery("SELECT c FROM Ciudad c WHERE c.id = :id" , Ciudad.class);
    Ciudad ciudad = query.setParameter("id", id).getSingleResult();
    em.remove(ciudad);
}
```

En el ejemplo se puede ver como, en primer lugar recuperamos la entidad a partir de su id para posteiormente ejecutar su borrado usando el método *remove*.

1.2.4.8. Propagación de Operaciones (Cascade)

Por defecto, las operaciones del Entity Manager se aplican únicamente a las entidades proporcionadas como argumento a sus diferentes métodos de operación. Esta operación, por

defecto, no se propagará a otras entidades que tienen relación con la entidad que se está modificando. Es bastante probable que si tenemos una entidad nueva y tiene una relación con otra entidad, en ambas se deba operar simultáneamente. El atributo Cascade nos permitirá la propagación en cascada de las operaciones realizadas en la entidad principal.

1.2.4.8.1. Cascade en JPA

Cascade, en JPA, **permite simplificar las operaciones en nuestro código** Java. Cuando realizamos alguna acción en la entidad objetivo, la misma acción se aplicará automáticamente a sus entidades asociadas.

Las relaciones entre entidades a menudo dependen de la existencia de otra entidad. Esto resulta directamente del mapeo realizado a la base de datos relacional en la que el ORM esté actuando.

Por ejemplo, en una relación Persona-Direcciones, si eliminamos la Persona, las entidades Dirección no tienen ningún sentido. Cuando eliminamos la entidad Persona, nuestras entidades Direcciones también deberían eliminarse. En otro tipo de relaciones similares, por ejemplo Persona-Ciudad debemos tener más cuidado, eliminar una Persona NO debería resultar en la eliminación de la Ciudad, ya que puede estar siendo usado por otras entidades de tipo Persona.

Las operaciones en cascada de JPA, representadas en el enum **javax.persistence.CascadeType**, son:

- **CascadeType.ALL**: se aplican todos los tipos de cascada.
- **CascadeType.PERSIST**: las operaciones de guardado en la base de datos de las entidades padre se propagarán a las entidades relacionadas.
- **CascadeType.MERGE**: las entidades relacionadas se unirán al contexto de persistencia cuando la entidad propietaria se una.
- **CascadeType.REMOVE**: las entidades relacionadas se eliminan de la base de datos cuando la entidad propietaria se elimine.
- **CascadeType.REFRESH**: las entidades relacionadas actualizan sus datos desde la base de datos cuando la entidad propietaria se actualiza.
- **CascadeType.DETACH** : se separan del contexto de persistencia todas las entidades relacionadas cuando ocurre una operación de separación manual.

Nota

Por defecto, no se aplica ninguna operación de cascada.

En este caso, el bloque de código SIN el atributo de Cascade sería:

```
Direccion direccion = new Direccion();
ArrayList<Direccion> direcciones = new ArrayList<Direcciones>();
direcciones.add(direccion)
Persona persona = new Persona();
persona.setDirecciones(direcciones);
entityManager.persist(direccion);
entityManager.persist(persona);
```

Y, aplicando **Cascade** a las entidades Direccion:

```
@Entity
public class Persona {
   // ...
   @OneToMany(cascade={CascadeType.PERSIST, CascadeType.REMOVE}, mappedBy = "persona", fetch =
 FetchType.LAZY)
   private List<Direccion> direcciones;
   @ManyToOne
   Ciudad ciudad;
   // ...
}
```

En el código anterior bastaría con realizar *entityManager.persist(persona);* ya que el resto de entidades también ejecutarían la orden persist al estar en cascada. Si realizamos un *entityManager.remove(persona);* entonces se eliminaría las direcciones asociadas.

```
Direccion direccion = new Direccion();
ArrayList<Direccion> direcciones = new ArrayList<Direcciones>();
direcciones.add(direccion)
Persona persona = new Persona();
persona.setDirecciones(direcciones);
entityManager.persist(persona);
```

1.2.4.8.1.1. ¿Qué atributos Cascade aplicar en nuestro modelo?

Para un mismo tipo de relación caben muchas posibilidades según los requisitos funcionales del modelo de negocio. No hay una norma escrita para aplicar **Cascade**, aunque la recomendación general es tener mucho cuidado con las operaciones en cascada en relaciones *many-to-many* (*@ManyToMany*) y en entidades compartidas. Cascade es una herramienta más de JPA, y es tarea del programador saber acertar en su uso. **Cascade** puede tanto facilitarnos la tarea como amargarnos la existencia si no se usa correctamente. Para un mismo tipo de relación caben muchas posibilidades. La respuesta es que **depende de la funcionalidad del modelo** de nuestra relación.

1.2.4.8.1.1.1. CascadeType.ALL

Por norma general el atributo **ALL** se debe intentar evitar a toda costa. La replicación de todas las operaciones en cascada puede jugar en nuestra contra si no sabemos el efecto de las mismas en nuestro modelo.

1.2.4.8.1.1.1.1. ¿Cuándo utilizar CascadeType.ALL?

En aquellos casos en que aplique el uso de todas las operaciones en cascada. Simplemente nos da legibilidad al código y permite ahorrarnos anotar todos los atributos. No se debe utilizar como comodín ni ponerse en casos de desconocimiento de la funcionalidad de la relación.

1.2.4.8.1.1.2. CascadeType.PERSIST

El atributo **PERSIST** propaga la operación de persistencia de una nueva relación principal a sus nuevas relaciones secundarias. Cuando guardamos la entidad *Persona*, la entidad de *Direccion* también se guardará.

1.2.4.8.1.1.2.1. ¿Cuándo utilizar CascadeType.PERSIST?

Es útil en los casos en los que sabemos que al insertar la entidad principal sus secundarias también han de ser insertadas porque se han creado en el mismo momento. Por ejemplo, si al crear una *Persona* también hemos creado su *Direccion*.

1.2.4.8.1.1.3. CascadeType.MERGE

El atributo **MERGE** propaga la operación de actualización en la base de datos de una relación principal a sus secundarias. Cuando guardamos la entidad Persona, la entidad de Direccion también se guardará.

1.2.4.8.1.1.3.1. ¿Cuándo utilizar CascadeType.MERGE?

Es útil en los casos en los que sabemos que al modificar la entidad principal sus secundarias también han de ser modificadas porque se han creado en el mismo momento. Por ejemplo, si al modificar una Persona también hemos modificado su Direccion.

1.2.4.8.1.1.4. CascadeType.REMOVE

El atributo **REMOVE** permite eliminar automáticamente todas las relaciones asociadas a la relación principal cuando ésta se elimina.

1.2.4.8.1.1.4.1. ¿Cuándo utilizar CascadeType.REMOVE?

A priori, la utilización del borrado en cascada, puede parecer habitual. Dependiendo de la complejidad de la relación se podría eliminar la necesidad de ir eliminando las instancias de otras entidades asociadas de forma programática. Sin embargo, aunque es un elemento muy interesante y facilitador, debe utilizarse con mucho cuidado.

Hay sólo dos situaciones en las que un borrado en cascada se puede usar sin problemas: relaciones *one-to-one* y *one-to-many*, en donde hay una clara relación de propiedad y la eliminación de la instancia principal debe causar la eliminación automática de sus instancias dependientes.

No se debe utilizar el borrado en cascada con relaciones múltiples *many-to-many*. Una colección anotada con @ManyToMany asocia dos entidades principales a través de una tabla de combinación, ¡no debemos ni queremos propagar la eliminación de un elemento principal a otro!.

Nota

No puede aplicarse ciegamente a todas las relaciones *one-to-one* o *one-to-many* porque las entidades dependientes podrían también estar participando en otras relaciones o podrían tener que mantenerse en la base de datos como entidades aisladas por requisitos funcionales.

1.2.4.8.1.1.5. CascadeType.REFRESH

El atributo **REFRESH** y el atributo **DETACH** no suelen ser habituales. Por tanto nos debemos centrar en el resto de atributos de Cascade.

1.2.4.8.1.1.5.1. ¿Cuándo utilizar CascadeType.REFRESH?

En el caso del atributo **REFRESH**, si realizamos una operación *EntityManager.refresh(objeto)* los objetos se vuelven a cargar desde la base de datos. El contenido del objeto gestionado en la memoria se descarta (incluidos los cambios, si los hubiera) y se reemplaza por los datos que se recuperan de la base de datos. Esto podría ser útil para garantizar que la aplicación maneje

la versión más actualizada de un objeto de entidad, en caso de que otro *EntityManager* lo haya cambiado desde que se recuperó. No es algo común en aplicaciones sencillas.

1.2.4.8.1.1.6. CascadeType.DETACH

1.2.4.8.1.1.6.1. ¿Cuándo utilizar CascadeType.DETACH?

El uso del atributo **DETACH** tampoco suele ser de uso común. En el caso de sacar un objeto del contexto de persistencia, de forma manual, entonces sí que deberíamos realizar la operación en cascada para las entidades relacionadas con el mismo. La operación *EntityManager.detach(objeto)* sólo la saca al objeto del contexto de persistencia, con lo que se cancelan los cambios al no persistirse en base de datos.

Las operaciones en cascada son unidireccionales. Debemos tener en cuenta cual es la entidad propietaria de la relación y dónde se va a actualizar la misma antes de tomar la decisión de poner el elemento en ambos lados. Por ejemplo, cuando definimos una nueva persona y una nueva dirección pondremos la dirección en el listado de direcciones de la persona. El atributo CascadeType se define únicamente en la relación Persona.

1.2.4.9. Repositorios JPA Predefinidos

Hasta el momento, hemos definido nuestro propio repositorio sin hacer uso de ninguna funcionalidad predefinida por Spring Data JPA. Sin embargo, y como es costumbre en todo el ecosistema Spring, lo normal es hacer uso de todas aquellas funcionalidades y herramientas que Spring Data JPA nos brinda y que facilitan y agilizan nuestro proceso de desarrollo.

La forma de hacer uso de las funcionalidades de repositorio ofrecidas por Spring Data JPA es mediante la definición de nuestro repositorio como una interfaz que herede de **Repository**. La tipamos con la clase del objeto del dominio que gestiona el repositorio, en nuestro caso **Ciudad**, y la de su identificador.

Se ve más claro con el siguiente ejemplo:

```
@Repository
public interface CiudadRepository extends org.springframework.data.repository.Repository<Ciudad.class,
 Long.class> {

}
```

Nota

Hay que especificar el paquete org.springframework.data.repository.Repository para que no haya problemas con la anotación @Repository

El propósito de la interfaz es informar a Spring que las interfaces que la especialicen representan repositorios. No se requiere nada más, como aplicar alguna anotación o seguir una convención a la hora de nombrar los repositorios.

Como alternativa, podemos definir un repositorio anotando una interfaz con *@RepositoryDefinition*:

```
@RepositoryDefinition(domainClass = Ciudad.class, idClass = Long.class)
public interface CiudadRepository {
}
```

Spring Data es capaz de encontrar y configurar todos los repositorios, siempre y cuando esté activada esa opción. Es el caso de Spring Boot y, por ende, del proyecto de ejemplo. Cuando no sea así, activaremos la detección y configuración de los repositorios marcando con *@EnableJpaRepositories* una clase de configuración (clase marcada con *@Configuration* o *@SpringBootApplication*). Por omisión, Spring buscará los repositorios a partir del paquete de la clase con esa anotación.

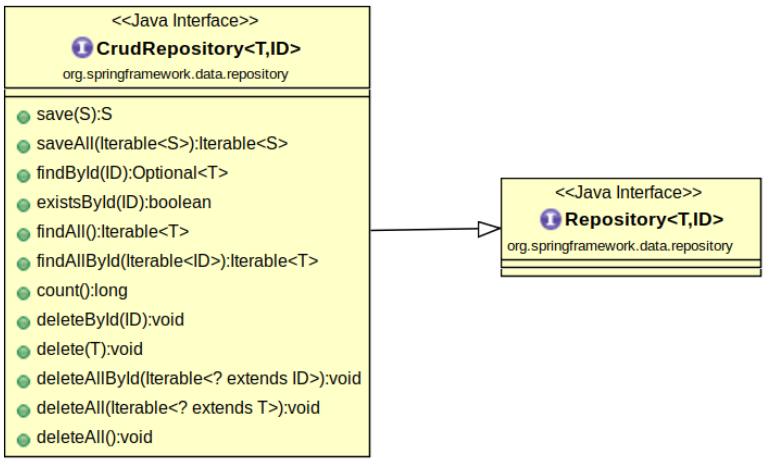
Pero, si el repositorio es una interfaz, ¿dónde la implementamos? En ningún sitio. Spring genera en tiempo de ejecución una implementación como un objeto *proxy* [https://es.wikipedia.org/wiki/Proxy_(patr%C3%B3n_de_dise%C3%B1o)].

Sea como fuere, *CiudadRepository* no sirve de nada, pues carece de métodos. A continuación dedicaremos varias secciones a revisar los repositorios genéricos que Spring Data y Spring Data JPA ofrecen de serie.

1.2.4.9.1. CrudRepository

Será habitual que muchas entidades precisen de un conjunto de operaciones denominadas CRUD: creación, lectura, actualización y borrado. El acrónimo viene del inglés (*Create*, *Read*, *Update*, *Delete*)

Spring Data proporciona un subtipo de *Repository* con el elocuente nombre de **CrudRepository**. Define doce métodos que, para mayor seguridad, respetan el tipado de la entidad y de su identificador.



La interfaz *CrudRepository* ofrece los siguientes métodos de lectura:

Optional<T> findById(ID id)	Obtiene una entidad dado su identificador. Cuando no exista devuelve un opcional vacío.
boolean existsById(ID id)	Indica si una entidad existe. Si solo queremos comprobarlo y no necesitamos la entidad, *existsById* es más legible y eficiente que *findById*.
Iterable<T> findAll()	Recupera todas las entidades. Usar este método es **peligroso** porque si hay muchas nos traeremos una colección inmensa de objetos.

	Debemos obtener las entidades en pequeños lotes con paginación, tal y como analizaremos en el capítulo nueve.
Iterable<t> findAllById(Iterable<T> ids)	Obtiene un lote de entidades a partir de sus identificadores únicos (no se admiten nulos, se lanzará *NullPointerException*). Las que no existan no aparecerán en los resultados, ni siquiera como nulas.
long count()	Cuenta el número total de entidades. Espero que nadie llame a *findAll* y cuente #

Estos son los métodos de escritura:

<S extends T> S save (S entity)	Guarda la entidad, tanto si es de nueva creación como si ya existe.
<S extends T> Iterable<S> saveAll(Iterable<S> entities)	Guarda las entidades solicitadas. En el caso de JPA, se llama a *save* para cada una de ellas.
void deleteById(ID id)	Elimina la entidad según su identificador. Si no existe, se lanza *EmptyResultDataAccessException*. Deberíamos ejecutar antes *existsById* para chequear la existencia. O mejor todavía, recurrir a otras alternativas que se propondrán en el curso.
void delete(T entity)	Elimina la entidad. Si no existe, no hace nada.
void deleteAllById(Iterable<? extends ID> ids)	Elimina las entidades indicadas con sus identificadores únicos (no se admiten nulos, se lanzará *NullPointerException*). La implementación de JPA invoca *deleteById* para cada identificador.
void deleteAll(Iterable<? extends T> entities)	Elimina las entidades recibidas como argumentos (no se admiten nulos). En JPA se ejecuta *delete* para cada identificador.
void deleteAll()	¡Lo borra todo! Además de **peligrosa**, es una operación ineficiente si hay muchas entidades porque las obtiene con *findAll* y luego llama a *delete* para cada una.

Una vez aquí, ¿cómo disfrutamos de los métodos de *CrudRepository?* Tan fácil como heredar de ella:

```
@Repository
public interface CiudadRepository extends CrudRepository<Ciudad.class, Long.class> {

}
```

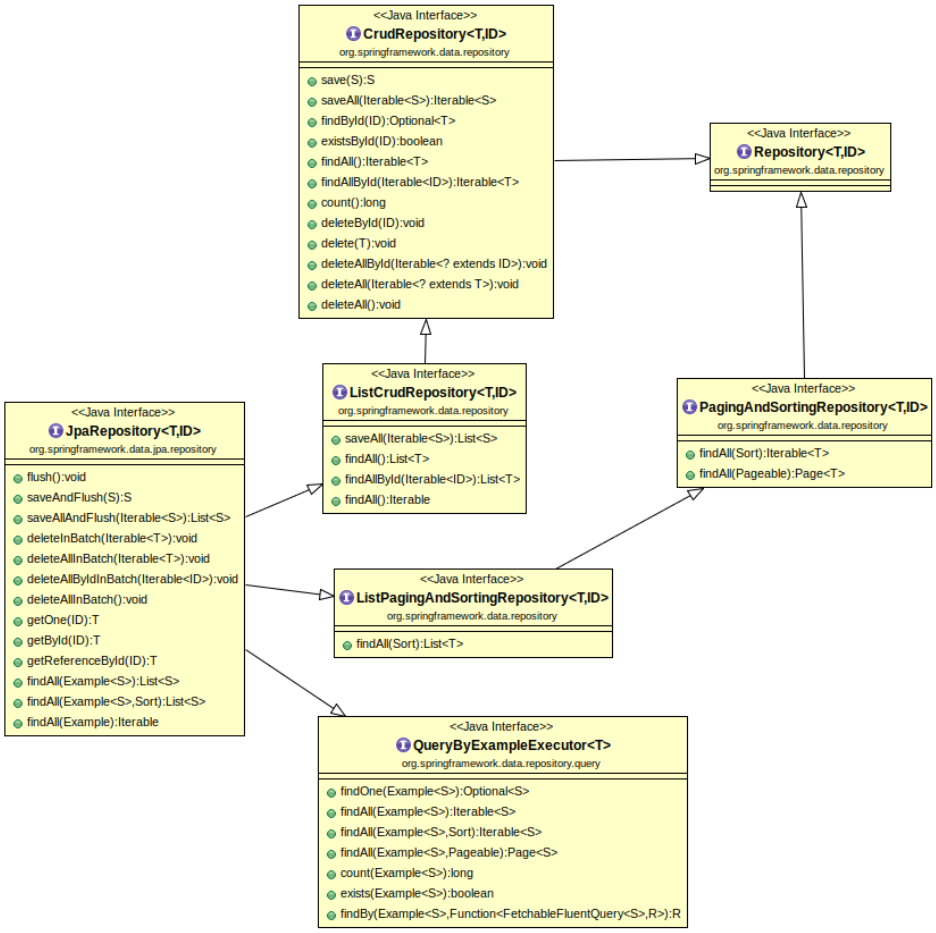
A partir de este momento, ya podremos hacer uso de todos los métodos ofrecidos por el repositorio genérico sin necesidad de escribir ni una línea de código.

```
@Autowired
private CiudadRepository ciudadRepository;

@Test
void testCreate() {
    Ciudad ciudad = new Ciudad();
    ciudad.setCodigo("MAL");
    ciudad.setDescripcionn("Málaga");
    ciudadRepository.save(country);
}
```

1.2.4.9.2. JPARepository

Los módulos de Spring Data suelen incluir repositorios genéricos con funcionalidades específicas de la tecnología que tratan. En Spring Data JPA encontramos **JpaRepository**.



JpaRepository es un tipo de **ListCrudRepository** con métodos adicionales. Varios de ellos deben obviarse porque están marcados obsoletos. También hereda de **ListPagingAndSortingRepository** y **QueryByExampleExecutor**. Dado que trataremos ambas interfaces en futuros apartados, vamos a ignorarlas de momento.

JpaRepository pone a nuestra disposición un gran número de métodos entre los cuales, y por su singularidad, podemos destacar el método *getReference*. Este método permite recuperar una referencia a una entidad a partir de su identificador:

La «referencia» retornada consiste en un objeto *proxy* que representa a la entidad y que solo contiene su identificador. Por tanto, la creación de la referencia no implica una consulta a la base

de datos que obtenga todos los datos de la entidad. Eso sí, cuando pidamos al *proxy* cualquier atributo distinto del identificador, se obtendrá el registro desde la tabla (lanzará una query).

Estamos ante una operación útil para afinar el rendimiento en ciertos escenarios. El más evidente que se me ocurre es el establecimiento de una relación simple, como la que se muestra a continuación:

```
...
Ciudad ciudad = ciudadRepository.findById(id).get();
persona.setCiudad(ciudad);
...
```

Date cuenta de que solo queremos la entidad representada por el objeto *ciudad* para relacionarla con *persona*. A su vez, lo único que JPA precisa de *ciudad* es su identificador porque será la clave ajena a la tabla *persona* del nuevo registro. Por este motivo es un desperdicio recuperar la entidad *ciudad* completa desde la base de datos con *findById*, acción que implica ejecutar una *SELECT*. Es suficiente con el objeto «referencia» que contiene el identificador:

```
...
Ciudad ciudad = ciudadRepository.getReference(id);
persona.setCiudad(ciudad);
...
```

Otro método importante es el método *flush*. La operación *flush* del gestor de entidades fuerza la sincronización inmediata de las entidades del contexto de persistencia con la base de datos. Será raro que tengamos que recurrir a este método, aún así, es importante conocer su existencia así como su función.

void flush()	Es lo que parece: invoca al método `flush`.
<S extends T> List<S> saveAll(Iterable<S> entities)	Llama a `save` para cada entidad.
<S extends T> List<S> saveAllAndFlush(Iterable<S> entities)	Invoca `saveAll` para aplicar `save` a cada entidad. Tras ello, llama a `flush`.

Otro grupo de métodos de **JpaRepository** son las operaciones de borrado en lote (batch):

```
void deleteAllInBatch();
void deleteAllInBatch(Iterable<T> entities);
void deleteAllInBatch();
```

Parecen más sofisticadas de lo que son. Borran las entidades en una sola sentencia *DELETE* de SQL. Si bien esto es más rápido que hacerlo de una en una llamando al *remove* del gestor de entidades, hay un inconveniente que puede ser importante: las entidades que ya estuvieran en el contexto de persistencia no se ven afectadas por el borrado.

Tampoco se propaga la operación de borrado hacia las entidades relacionadas aunque así lo hayamos establecido en la configuración de la clase de las entidades que estamos eliminando (CascadeTyple.REMOVE).

1.2.4.9.3. Personalizar los repositorios genéricos

A priori, las operaciones de los repositorios genéricos como **JpaRepository** nos vienen muy bien. El problema es que cuando heredamos de **JpaRepository**, o de cualquier interfaz, es

imposible excluir métodos; es todo o nada. En consecuencia, es posible que agreguemos a nuestros repositorios operaciones innecesarias o, peor aún, que no queremos.

Por fortuna, podemos incorporar en un repositorio métodos de los repositorios genéricos con tan solo copiar su signatura. En realidad estamos declarando «consultas derivadas», que veremos en el siguiente apartado.

Imagina que deseamos que el repositorio de las *Ciudades* ofrezca algunos métodos de consulta de **JpaRepository**. No queremos el resto, luego heredar de **JpaRepository** no es una opción. Así que heredamos de **Repository** y copiamos los métodos de JpaRepository que necesitemos:

```
public interface CiudadJpaRepository extends Repository<Ciudad, Long> {

    Optional<Ciudad> findById(Long id);

    boolean existsById(Long id);

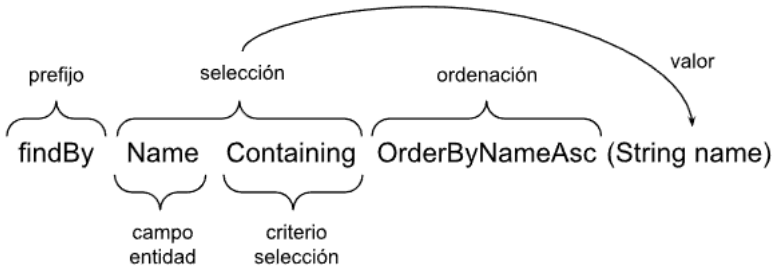
    long count();

}
```

1.2.4.10. Consultas Derivadas (Derived Queries)

Las consultas derivadas es posiblemente la funcionalidad más llamativa de Spring Data. Las consultas derivadas (*derived queries*) definen consultas para la clase de dominio que gestiona el repositorio con el nombre del propio método. Con otras palabras, la consulta se "deriva» o deduce del nombre del método. Todo ello desenvoca en consultas fáciles de escribir, pero también de leer.

Ese nombre se compone de un prefijo que indica el tipo de consulta, seguido de los criterios de selección de los resultados y, con carácter opcional, de la ordenación. Así, *findByNombreContainingOrderByNameAsc* busca los países (*find..By*) cuyo atributo nombre (*Nombre*) contenga (*Containing*) cierta cadena (el parámetro del método). Los resultados se ordenan (*OrderBy*) por el campo nombre (*Nombre*) de forma ascendente (*Asc*).



Las búsquedas se indican con los prefijos *find...By*, *read...By*, *get...By*, *query...By*, *search...By* o *stream...By*. Todos hacen lo mismo. La elección dependerá del significado que quieras darle al método.

Los siguientes métodos son equivalentes:

```
List<Persona> findByNombreContainingOrderByNameAsc(String nombre);
List<Persona> readByNombreContainingOrderByNameAsc(String nombre);
List<Persona> getByNombreContainingOrderByNameAsc(String nombre);
List<Persona> queryByNombreContainingOrderByNameAsc(String nombre);
List<Persona> streamByNombreContainingOrderByNameAsc(String nombre);
List<Persona> searchByNombreContainingOrderByNameAsc(String nombre);
```

Lo que escribas en los “tres puntos» de los prefijos se ignora, con la excepción de las palabras *Distinct*, *Top* y *First* que ya veremos. Por ello, el siguiente método equivale a cualquiera de los anteriores:

```
List<Persona> findPersonasByNombreContainingOrderByNameAsc(String nombre);
```

Cuando Spring Boot arranque y cree las implementaciones de cada repositorio también generará una consulta JPQL para cada método del repositorio. Si no fuera posible (query errónea), lanzará una excepción de tipo **QueryCreationException** con un mensaje indicando la naturaleza del problema.

1.2.4.10.1. Las palabras claves para la definición de Consultas

En este apartado vamos a estudiar las palabras claves (*keywords*) disponibles para construir las expresiones lógicas de selección de entidades. Para que la explicación sea más instructiva vamos a apoyarnos en una serie de métodos que realizan una consulta sobre una entidad *Persona* y mostraremos la traducción a SQL que realizará JPA.

La entidad *Persona* se corresponde con la siguiente clase:

```
@Entity
@Table(name = "persona")
public class Persona {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "ID")
    private Long id;

    @Column(name = "NOMBRE")
    private String nombre;

    @Column(name = "APELLIDO1")
    private String apellido1;

    @Column(name = "APELLIDO2")
    private String apellido2;

    @Column(name = "FECHA_NACIMIENTO")
    private Date fechaNacimiento;

    @Column(name = "PESO")
    private Float peso;

    @Column(name = "ALTURA")
    private Double altura;

    @Column(name = "ACTIVO")
    private Boolean activo;

    //RELACIONES
    @OneToOne(optional = true, fetch = FetchType.LAZY)
    @JoinColumn(name = "OPERADOR", referencedColumnName = "ID", nullable = true)
    private Operador operador;

    @ManyToOne(optional = false, fetch = FetchType.LAZY)
    @JoinColumn(name = "CIUDAD", referencedColumnName = "ID", nullable = true)
    private Ciudad ciudad;

    @OneToMany(mappedBy = "persona", fetch = FetchType.LAZY)
    private List<Direccion> direcciones;

    @JoinTable(name = "REL_PERSONA_HOBBY", joinColumns = @JoinColumn(name = "PERSONA", nullable = false),
 inverseJoinColumns = @JoinColumn(name = "HOBBY", nullable = false))
    @ManyToMany(fetch = FetchType.LAZY)
    private List<Hobby> hobbies;

    ...

}
```

1.2.4.10.1.1. Selección de resultados

Equals o **Is**. Condición de igualdad.

"Buscar personas cuya fecha de nacimiento coincida con la recibida como parámetro"

```
List<Persona> findByFechaNacimientoEquals(Date fechaNacimiento);
List<Persona> findByFechaNacimientoIs(Date fechaNacimiento);
```

```
where p1_0.fecha_nacimiento=?
```

Ninguna keyword. Equivale a **Equals** o **Is**. La siguiente consulta es igual que la anterior:

```
List<Persona> findByFechaNacimiento(Date fechaNacimiento);
```

```
where p1_0.fecha_nacimiento=?
```

Not, **IsNot**. La desigualdad.

"Buscar personas cuya fecha de nacimiento no coincida con la recibida como parámetro"

```
List<Persona> findByFechaNacimientoNot(Date fechaNacimiento);
List<Persona> findByFechaNacimientoIsNot(Date fechaNacimiento);
```

```
where p1_0.fecha_nacimiento!=?
```

After, **IsAfter**. Útil para filtrar por fechas posteriores (o estrictamente mayores) a la indicada.

"Buscar personas con fecha de nacimiento posterior a la fecha recibida como parámetro"

```
List<Persona> findByFechaNacimientoAfter(Date fechaDesde);
List<Persona> findByFechaNacimientoIsAfter(Date fechaDesde);
```

```
where p1_0.fecha_nacimiento>?
```

Before, **isBefore**. Ahora filtramos por una fecha anterior (estrictamente menor) a la indicada.

"Buscar personas con fecha de nacimiento posterior a la fecha recibida como parámetro"

```
List<Persona> findByFechaNacimientoBefore(Date fechaHasta);
List<Persona> findByFechaNacimientoIsBefore(Date fechaHasta);
```

```
where p1_0.fecha_nacimiento<?
```

Between, **isBetween**. El operador **BETWEEN** de JPQL\SQL. Los dos extremos del rango se incluyen.

"Buscar personas con fecha de nacimiento entre las fechas recibidas como parámetro"

```
List<Persona> findByFechaNacimientoBetween(Date fechaDesde, Date fechaHasta);
```

```
where p1_0.fecha_nacimiento between ? and ?
```

LessThan, **isLessThan**. Menor estricto.

"Buscar personas con altura menor a la recibida como parámetro"

```
List<Persona> findByAlturaLessThan(Double altura);
List<Persona> findByAlturaIsLessThan(Double altura);
```

```
where p1_0.altura<?
```

El menor o igual es **LessThanEqual**, **IsLessThanEqual**.

GreaterThan, **IsGreaterThan**. Mayor estricto.

"Buscar personas con altura mayor a la recibida como parámetro"

```
List<Persona> findByAlturaGreaterThan(Double altura);
List<Persona> findByAlturaIsGreaterThan(Double altura);
```

```
where p1_0.altura>?
```

El mayor o igual es **GreaterThanEqual**, **IsGreaterThanEqual**.

Null, IsNull. Imponen la condición de que una propiedad sea nula.

"Buscar personas cuyo peso es nulo"

```
List<Persona> findByPesoNull();
List<Persona> findByPesoIsNull();
```

```
where p1_0.peso is null
```

NotNull, **IsNotNull**. La propiedad no puede ser nula.

"Buscar personas cuyo peso es distinto de nulo"

```
List<Persona> findByPesoNotNull();
List<Persona> findByPesoIsNotNull();
```

```
where p1_0.peso is not null
```

Like, **IsLike**. El operador *LIKE* para cadenas de JPQL.

"Buscar personas cuyo primer apellido coincida con la cadena recibida como parámetro"

```
List<Persona> findByApellido1Like(String cadena);
List<Persona> findByApellido1IsLike(String cadena);
```

```
where p1_0.apellido1 like ?
```

Se niega con **NotLike**, **IsNotLike**.

Containing, **IsContaining**, **Contains**. Aplica el operador *LIKE* envolviendo la cadena a buscar con el comodín *%*.

"Buscar personas cuyo primer apellido contenga la cadena recibida como parámetro"

```
List<Persona> findByApellido1Containing(String cadena);
List<Persona> findByApellido1IsContaining(String cadena);
List<Persona> findByApellido1IsContain(String cadena);
```

```
where p1_0.apellido1 like ?
```

Se niega con **NotContaining**, **IsNotContaining**.

StartingWith, **StartsWith, IsStartingWith**. La cadena debe comenzar por cierto prefijo. Aplica *LIKE* añadiendo al argumento el marcador *%* al final.

"Buscar personas cuyo primer apellido comience con la cadena recibida como parámetro"

```
List<Persona> findByApellido1StartingWith(String cadena);
List<Persona> findByApellido1StartsWith(String cadena);
List<Persona> findByApellido1IsStartingWith(String cadena);
...
personaRepository.findByApellido1IsStartingWith("García");
```

```
where p1_0.apellido1 like ?
```

JPA hace la traducción automática del parámetro:

```
binding parameter [1] as [VARCHAR] - [García%]
```

EndingWith, **IsEndingWith**, **EndsWith**. La cadena debe terminar con cierto sufijo. Aplica *LIKE* agregando al argumento el marcador *%* al principio.

"Buscar personas cuyo primer apellido finalice con la cadena recibida como parámetro"

```
List<Persona> findByApellido1EndingWith(String cadena);
List<Persona> findByApellido1EndsWith(String cadena);
List<Persona> findByApellido1IsEndingWith(String cadena);
...
personaRepository.findByApellido1IsEndingWith("García");
```

```
where p1_0.apellido1 like ?
```

JPA hace la traducción automática del parámetro:

```
binding parameter [1] as [VARCHAR] - [%García]
```

In, **IsIn**. El operador lógico *IN* aplicado a un conjunto de valores. Como parámetro del método se admite cualquier *Collection*, un *array* o un *varargs*.

"Buscar personas que tengan asociada una ciudad cuyo id esté contenido en el listado recibido como parámetro"

```
List<Persona> findByCiudadIdIn(Collection<Long> ids);
```

```
where p1_0.ciudad_id in(?,?)
```

Fíjate que esta consulta selecciona los resultados por una propiedad de una relación (ciudad) en lugar de por un atributo. **Se permite navegación por las relaciones**.

Se niega con **NotIn**, **IsNotIn**.

True, **False**, **IsTrue**, **IsFalse**. Establece el valor de una propiedad lógica.

"Buscar personas que estén activos"

```
List<Persona> findByActivoTrue();
List<Persona> findByActivoIsTrue();
List<Persona> findByActivoFalse();
List<Persona> findByActivoIsFalse();
```

```
where not(p1_0.activo)
where p1_0.activo
```

IgnoreCase, IgnoringCase. Compara cadenas con el operador = ignorando la capitalización (mayúsculas y minúsculas) gracias al empleo de la función *UPPER*.

"Buscar personas cuyo nombre coincida con el recibido como parámetro ignorando mayúsculas y minúsculas"

```
List<Persona> findByNombreIgnoreCase(String nombre);
```

```
where upper(p1_0.nombre)=upper(?)
```

Estas palabras clave son combinables con *containing*, *startingWith* y *endingWith*.

Empty, **IsEmpty**. Aplica el operador *EMPTY* de JPQL a una propiedad de una clase de entidad que sea una colección.

"Buscar personas que no tengan direcciones asociadas"

```
List<Persona> findByDireccionesEmpty();
List<Persona> findByDireccionesIsEmpty();
```

1.2.4.10.1.2. Unión de Criterios

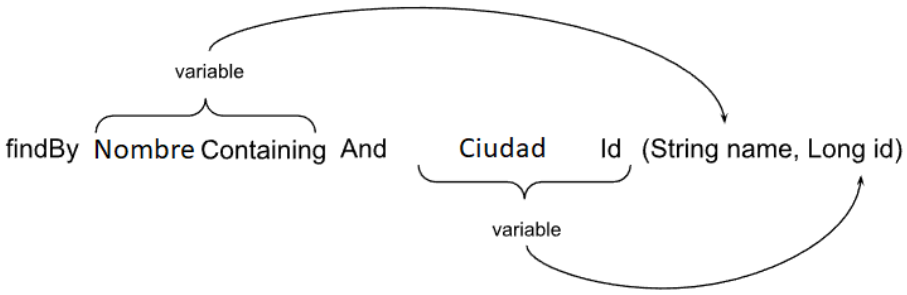
And une criterios.

"Todas las personas que contengan cierta cadena en su nombre y estén asociadas a cierta ciudad dada por su identificador"

```
List<Persona> findByNombreContainingAndCiudadId(String nombre, Long id)
```

```
where p1_0.name like ? and p1_0.ciudad_id=?
```

Hay que tener muy en cuenta que la posición de cada parámetro del método debe respetar la posición en la que aparece la variable a la que está vinculada en el nombre del método. Por ello, el método anterior recibe primero el nombre de la persona y luego el identificador de la ciudad. Debido a esta convención, el nombre de los parámetros resulta irrelevante.



Observa también que la unión de múltiples criterios alarga el nombre del método. Por paradójico que suene, un método con un nombre descriptivo pero extenso es difícil de leer.

Or es la condición *Or* de toda la vida.

"Todas las personas que contengan cierta cadena en su nombre o estén asociadas a cierta ciudad dada por su identificador"

```
List<Persona> findByNombreContainingOrCiudadId(String nombre, Long id)
```

```
where p1_0.name like ? or p1_0.ciudad_id=?
```

1.2.4.10.1.3. Ordenación

La expresión **OrderBy** equivale a la cláusula *ORDER BY* de JPQL. A ella le añades los atributos por los que efectuar la ordenación. Debes especificar para cada uno el sentido o dirección de ordenación: ascendente (**Asc**) o descendente (**Desc**).

"Todas las personas ordenadas según su fecha de nacimiento en sentido ascendente"

```
List<Persona> findByOrderByFechaNacimientoAsc()
```

Si el último atributo por el que ordenas tiene sentido ascendente, no hace falta que le agreges *Asc*.

```
List<Persona> findByOrderByFechaNacimiento()
```

Existe la posibilidad de ordernar por varios atributos a la vez.

"Todas las personas ordenadas según su fecha de nacimiento en sentido ascendente y por su nombre"

```
List<Persona> findByOrderByFechaNacimientoAscNombre()
```

```
order by p1_0.fecha_nacimiento asc,p1_0.nombre asc
```

El *Asc* que acompaña a *FechaNacimiento* es imprescindible porque separa la declaración de ese atributo de la declaración de *Nombre*. Sin ese *Asc*, el método no representa una consulta derivada válida. En el caso de *Name*, no requiere *Asc*, pues se trata del último atributo.

1.2.4.10.1.4. Limitar los Resultados

Es posible limitar el número de resultados a obtener indicando las palabras reservadas **First** o **Top** entre el prefijo de la consulta y *By*. Tras **First** o **Top**, se indica la cantidad máxima de resultados; si no se especifica, se asume **uno como valor por defecto**.

"Las tres personas más mayores"

```
List<Persona> findTop3ByOrderByFechaNacimientoAsc()
```

```
order by p1_0.fecha_nacimiento asc fetch first ? rows only
```

Si quieres quedarte con los resultados distintos en una consulta que pueda devolver duplicados, escribe **Distinct** entre *find...By*.

1.2.4.10.1.5. Consultas de Conteo y Existencia

La búsqueda de entidades con *find...By* y sus sinónimos se complementa con dos prefijos (**count…By** y **exists…By**), que admiten los mismos criterios de selección de entidades que *find...By*.

count…By. El resultado de la función agregada de conteo que permite contar elementos en una consulta.

"Número de personas que viven en una ciudad cuyo id se recibe como parámetro"

```
Long countByCiudadId(Long id);
```

```
select count(p1_0.id) from persona p1_0 where p1_0.ciudad_id=?
```

Ni que decir tiene que es más eficiente emplear *count...By* que obtener las entidades y contarlas.

exists…By. Indica la existencia de al menos un resultado.

"Comprobar si existe una persona con el nombre recibido como parámetro"

```
Boolean existsByNombreIgnoringCase(String nombre);
```

```
select p1_0.id from persona p1_0
where upper(p1_0.nombre)=upper(?) fetch first ? rows only
```

Spring Data JPA recupera el identificador del primer resultado. Por tanto, esta manera de comprobar la existencia de entidades resulta muy eficiente.

1.2.4.11. Consultas JPQL @Query

La técnica consistente en declarar una consulta con el nombre del método (Derived Query) es en sí misma una limitación: no podemos expresar demasiado, tan solo seleccionar entidades con criterios muy sencillos. Aunque a veces sea suficiente, la experiencia nos dice que la mayoría de los métodos de los repositorios no entran en esa categoría.

Otra desventaja de las consultas derivadas, menos evidente, es que resulta inaceptable crear métodos con nombres demasiado largos. Es lo que ocurrirá si acumulamos varios criterios de selección.

Desde el primer momento hemos defendido el uso de nombres claros, descriptivos y precisos, siguiendo las prácticas recomendadas por la filosofía del código limpio. Este tipo de nombres no solo facilitan la lectura y, por tanto, la comprensión del código, sino que además nos obligan a reflexionar sobre el cometido de una variable, una clase o un método.

Dicho esto, cuando la extensión de un nombre supera lo razonable conseguimos el efecto contrario, pues dificultará su lectura. Pecamos de ser más descriptivos de la cuenta.

Por último, y no menos decisivo, las consultas derivadas solo retornan entidades (ignorando las operaciones de tipo conteo y existencia) del tipo que maneje el repositorio o bien un conjunto

de sus atributos. No todas las consultas que necesitemos se amoldarán a este patrón por lo que tenemos que buscar una alternativa y esa alternativa es el uso de **@Query**.

El uso de la anotación nos permitirá hacer uso de las consultas JPQL sin necesidad de realizar la implementación del método correspondiente.

Vamos a intentar comprender su funcionamiento en base a un ejemplo anterior. Consideremos las siguiente consula derivada definida en el repositorio *PersonaRepository*:

```
List<Persona> findByFechaNacimientoEquals(Date fechaNacimiento);
```

Como podemos observar, dicha consulta busca todas las personas cuya fecha de nacimiento coincida con la que se recibe como parámetro. Esta consulta se puede implementar perfectamente usando una consulta JPQL equivalente tal que así:

```
SELECT p FROM Persona p WHERE p.fechaNacimiento = :fechaNacimiento
```

Pero ¿Dónde colocamos dicha consulta JPQL si nuestro repositorio es una interfaz donde no es posible implementar método alguno?, la respuesta es la anotación @Query. A continuación podemos observar cómo quedaría dicha consulta usando la consulta JPQL:

```
@Query ("SELECT p FROM Persona p WHERE p.fechaNacimiento = ?1")
List<Persona> findByFechaNacimientoEquals(Date fechaNacimiento);
```

Fíjate que el nombre *findByFechaNacimientoEquals* define una consulta derivada. Pero cuando el método está marcado con *@Query*, nunca se interpreta como una consulta derivada; así pues, el nombre es irrelevante.

El gran inconveniente de las cadenas es que lo aguantan todo: puedes escribir cualquier cosa en ellas. Spring Data JPA minimiza este riesgo chequeando al arrancar que todas las consultas JPQL son válidas. Si alguna fuera incorrecta, una excepción de tipo *QueryCreationException* detiene el inicio de la aplicación. El mismo mecanismo de seguridad se aplica a las consultas derivadas.

Ahora que tenemos una consulta JPQL vinculada a un método, veamos con detalle los posibles parámetros de ese método, así como su retorno.

1.2.4.11.1. Variables

Las variables se declaran en la consulta con los marcadores estándar de JPQL. En el ejemplo anterior, un interrogante seguido de un número «i» que indica que la variable se reemplazará con el valor del parámetro del método que ocupa la posición «i» (parámetros por posición). Sin embargo, el código queda más claro si usas como marcador el nombre del parámetro precedido por dos puntos (parametros nombrados):

```
@Query ("SELECT p FROM Persona p WHERE p.fechaNacimiento = :fechaNacimiento")
List<Persona> findByFechaNacimientoEquals(Date fechaNacimiento);
```

La vinculación entre marcador y parámetro es configurable con la anotación *@Param*, así que ambos pueden tener nombres diferentes, como muestra este ejemplo:

```
@Query ("SELECT p FROM Persona p WHERE p.fechaNacimiento = :fechaNacimiento")
List<Persona> findByFechaNacimientoEquals(@Param("fechaNacimiento") Date fecha);
```

En todos los casos, un mismo parámetro puede aparecer en la consulta como variable más de una vez. Con otras palabras, es reutilizable. Por el contrario, en las consultas derivadas cada variable se corresponde un parámetro distinto.

1.2.4.11.2. Respuesta

En cuanto a los posibles tipos de respuesta de los métodos anotados con *@Query*, nada cambia en lo que se refiere a las consultas derivadas. En esencia, devolvemos una entidad o un valor primitivo en un resultado único (por ejemplo, con *Optional* si puede ser *null*) o una colección (*List*, *Collection*, *Iterator*, etc).

1.2.4.11.3. Consultas nombradas (named queries)

Recordemos que las consultas nombradas de JPA son consultas JPQL a las que asignamos un nombre único que funciona a modo de identificador.

Las consultas nombradas ofrecen dos beneficios. En primer lugar, se procesan cuando el sistema arranca, de modo que si son incorrectas se producirá una excepción y el inicio de la aplicación se aborta. La segunda ventaja es que aportan una forma estándar de separar las cadenas JPQL del código Java.

Ambas características no son relevantes en Spring Data JPA porque ya las tenemos con *@Query*.

En caso de que la consulta que hemos visto anteriormente se encontrase definida como *NamedQuery* en la entidad *Persona*.

```
@NamedQuery(name = "Persona.findByFechaNacimiento",
        query = """
                SELECT p FROM Persona p
                WHERE p.fechaNacimiento = :fechaNacimiento""")
```

Su uso mediante *@Query* sería trivial ya que sólo deberíamos especificar su identificador en el atributo *name* de la anotación para que dicha consulta quedara asociada al método correspondiente.

```
@Query (name = "Persona.findByFechaNacimiento")
List<Persona> findByFechaNacimientoEquals(@Param("fechaNacimiento") Date fecha);
```

1.2.4.12. Ordenación

A priori, la ordenación de los resultados de las consultas parece un asunto menor y rápido de tratar. Sin embargo, se trata de un aspecto bastante importante cuando acometemos el desarrollo de una aplicación real.

En apartados anteriores han aparecido ejemplos de ordenación de resultados. Por ejemplo:

- Consulta derivada con expresiones.

```
List<Persona> findByOrderByFechaNacimientoAsc()
```

- Consulta JPQL con cláusula

```
@Query("SELECT p FROM Persona p ORDER BY p.fechaNacimiento ASC")
```

```
List<Persona> findByOrderByFechaNacimientoAsc()
```

En las consultas anteriores hay un detalle muy importante: la ordenación es estática. Se establece en la definición del método o consulta y no se puede cambiar.

Si necesitas aplicar distintos criterios de ordenación a una consulta, resulta tentador duplicarla y cambiar la ordenación. Para evitar esto, Spring Data te permite declarar criterios dinámicos, esto es, en tiempo de ejecución. Además, disfrutar de esta funcionalidad con todos los tipos de consulta admitidos por Spring Data JPA.

1.2.4.12.1. Ordenación con Sort

La ordenación dinámica se basa en el uso de la clase **Sort**, su uso es simple pero pontente. A continuación podemos observar un ejemplo de uso de Sort:

```
List<Persona> findByOrderByFechaNacimientoAsc(Sort sort)
```

Es lo que parece: se añade a la signatura del método un parámetro de tipo Sort con los criterios de ordenación.

1.2.4.12.1.1. Creación de objetos Sort básicos con by

A la hora de crear nuestro objeto Sort devemos olvidarnos de sus constructores, pues son privados. En su lugar, hay que recurrir a las sobrecargas del método estático **by**. Estas son las de uso más habitual:

- **public static Sort by(String... properties)** . Recibe un parametro con los nombres de los campos. El sentido de la ordenación es ascendente. Esto implica que la siguiente llamada equivale a invocar al método **findByOrderByFechaNacimientoAsc**.

```
Sort sortByFechaNacimiento = Sort.by("fechaNacimiento");
public List<Persona> findBy(Sort sort);
```

- **public staticSort by(Direction direction, String... properties)**. De nuevo, un método que recibe los campos en un parámetro , pero acompañado del sentido. Esta llamada equivale a la anterior en sentido inverso:

```
Sort sortByFechaNacimiento = Sort.by(Sort.Direction.DESC, "fechaNacimiento");
public List<Persona> findBy(Sort sort);
```

Los métodos **by** crean objetos **Sort** que ya contienen en su interior los objetos **Order** adecuados. La lista que los contiene respeta el orden de declaración de los campos, de tal modo que aparecerán en ese orden en la cláusula ORDER BY de la consulta SQL.

También es posible realizar la ordenación usando varios campos, incluyendo la posibilidad de establecer un sentido distinto a cada uno de ellos. Para ello la clase **Sort** incluye los siguientes métodos:

`Sort descending()`	Devuelve un nuevo **Sort** con los mismos criterios de ordenación que el objeto **sort** sobre

	el que se invoca el método, pero todos en sentido descendente.
`Sort ascending()`	Devuelve un nuevo **Sort** con los mismos criterios de ordenación que el objeto **sort** sobre el que se invoca el método, pero todos en sentido ascendente.
`Sort and(Sort sort)`	Devuelve un nuevo Sort con los mismos criterios de ordenación que el objeto **sort** sobre el que se invoca el método más los del argumento.

Los métodos **descending**, **ascending** y **and** devuelven nuevos objetos **Sort** que pueden ser encadenados para elaborar criterios de ordenación complejos.

```
Sort sort = Sort.by("fechaNacimiento").descending().and(Sort.by("nombre"));
```

```
Sort sort = Sort.by(Sort.Direction.DESC, "fechaNacimiento").and(Sort.by("nombre"));
```

En caso de no querer establecer criterio de ordenación alguno es posible usar **Sort.unsorted()**.

Adicionalmente a los mecanismos expuestos en este apartado para la ordenación, existen otras técnicas avanzadas que no estudiaremos durante este curso. Entre ellas se encuentran el uso de los objetos **Sort.Order**, la ordenación con funciones basadas en **JpaSort** o los Sort tipados (**TypedSort**).

1.2.4.12.1.1.1. Prioridad de ordenaciones

Llegados a este punto, nos puede surgir la duda de qué sucederá si aplicamos **Sort** a una consulta que ya tiene una ordenación declarada. Es decir, cómo se ejecuta la ordenación si aplicamos un **Sort** a una **@Query** o una **consulta derivada**.

La respuesta es simple, Spring Data JPA combina las dos ordenaciones, añadiendo la del objeto **Sort** a la establecida dentro de la propia consulta. Eso sí, cuando aparezcan criterios repetidos, las duplicidades no se eliminan.

1.2.4.13. Paginación

Cuanta más información recupere una consulta, mayor lentitud de la operación y consumo de memoria. El rendimiento del sistema se degrada y, en casos extremos, el sistema puede venirse abajo por un *OutMemoryException* o similar. Por ejemplo el método **findAll** de **JpaRepository** recupera todas las entidades de un tipo; si en la tabla hubiera miles de registros, **findAll** devolvería una lista gigantesca capaz de tumbar la aplicación.

Además, quizás este problema pase desapercibido durante los primeros meses de uso del sistema porque existan pocos registros en la base de datos. Pero a medida que el tiempo pase, las tablas aumentarán de tamaño y el rendimiento del sistema se irá degradando.

La solución a dicho problema es simple, no debemos recuperar todos los resultados de una tacada. Para ello es necesario agrupar los resultados en lotes (páginas) pequeños y manejables que se irán recuperando de la base de datos de forma progresiva a medida que sean necesarios.

La división en páginas debe realizarla la propia consulta en la base de datos, de forma que cada página se recupere ejecutando una nueva consulta.

A continuación podemos observar un ejemplo más que conocido por todos:



Otro ejemplo es el de aplicaciones para móviles en las que la información se muestra en un listado y las nuevas páginas se le van agregando con discreción a medida que lo deslizas.

Los elementos usados habitualmente en las aplicaciones web para implementar la paginación son las denominadas barras de paginación. Un ejemplo genérico de este tipo de componentes es la que podemos observar a continuación:

Este componente gráfico permite indicar el tamaño de cada página (el número de filas de la tabla) y navegar entre ellas con botones. Este tamaño tendrá un límite máximo razonable y es necesario asegurar en el código que sea respetado. Asimismo, podemos observar que un componente de este tipo precisa conocer el número total de resultados para calcular el total de páginas existentes.

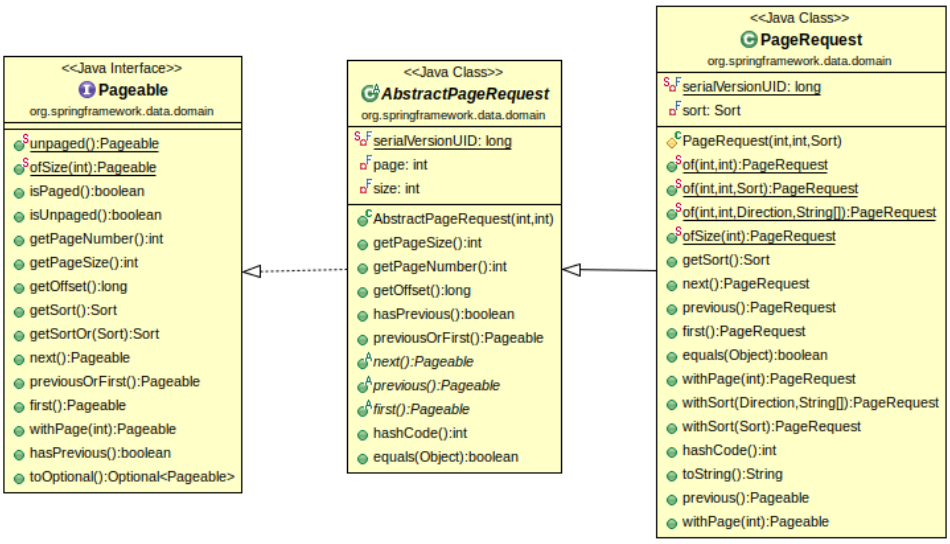
1.2.4.13.1. La interfaz Pageable

La paginación en **Spring Data JPA** se implementa de forma análoga a la usada para la ordenación dinámica vía objetos **Sort**. Es necesario añadir al método un parámetro de cierto tipo. Si en el caso de la ordenación ese parámetro es de la clase **Sort**, para la paginación es la interfaz **Pageable**, la cual modela la configuración que Spring Data precisa para paginar los resultados.

Aquí podemos observar un ejemplo de su uso:

```
List<Persona> findAll(Pageable pageable);
```

La interfaz **Pageable** requiere de una implementación. Spring Data la provee con la clase **PageRequest**.



PageRequest se apoya en tres atributos para cumplir con el contrato definido por la interfaz:

- **page**: El número de la página. Empieza en cero.
- **size**: El tamaño máximo de la página; es decir, los resultados que contiene.
- **sort**: Los criterios de ordenación.

Al igual que en el caso de los objetos Sort, la creación de los objetos **PageRequest** se realizan mediante el uso de varios constructores estáticos. En esta ocasión, los métodos tienen el prefijo **of** en lugar de **by**.

El más simple (método **ofSize**) solo requiere el tamaño de la página, la cual se refiere a la primera (**la cero**):

```
public static PageRequest ofSize(int pageSize) {
    return PageRequest.of(0, pageSize);
}
```

Aunque existen varios más:

```
public static PageRequest of(int page, int size) {
  return of(page, size, Sort.unsorted());
}

public static PageRequest of(int page, int size, Sort sort) {
  return new PageRequest(page, size, sort);
}

public static PageRequest of(int page, int size, Direction direction, String... properties) {
    return of(page, size, Sort.by(direction, properties));
}
```

Aunque es posible crear objetos **Pageable** sin ordenación, es indispensable asegurarnos de que la consulta aplique un orden. En aquellos casos en los que no existe un orden a aplicar habrá que establecer una ordenación predeterminada que tenga sentido para los datos con los que opera el sistema.

En el siguiente ejemplo la propia consulta ya contiene una ordenación. Si **pageable** también tuviera una ordenación, se agregará a la de la consulta.

```
List<Persona> findAllByFechaNacimiento(Pageable pageable);
```

Cabe destacar que esto es incorrecto:

```
List<Persona> findAll(Pageable pageable, Sort sort);
```

El método lanzará una excepción, cuyo mensaje explica que está prohibido declarar un parámetro **Sort** y otro **Pageable** en un mismo método. En vez de ello, cuando la ordenación sea dinámica debes incluirla en el objeto **Pageable** con los métodos **of** adecuados.

Un detalle muy importante a la hora de trabajar con búsquedas paginadas es el cálculo de las páginas existentes. O lo que es lo mismo, la cantidad total de resultados. Ese dato es indispensable para facilitar la navegación por los resultados y, por ejemplo, construir una barra de navegación como la que hemos visto anteriormente.

El conteo de los resultados de una consulta se puede obtener escribiendo una consulta derivada cuyo nombre comience con la expresión *count...By*:

```
Long countPersonaByCiudadId(Long ciudadId);
```

En JPQL, igual que en SQL, se usa la función de agregación *COUNT*:

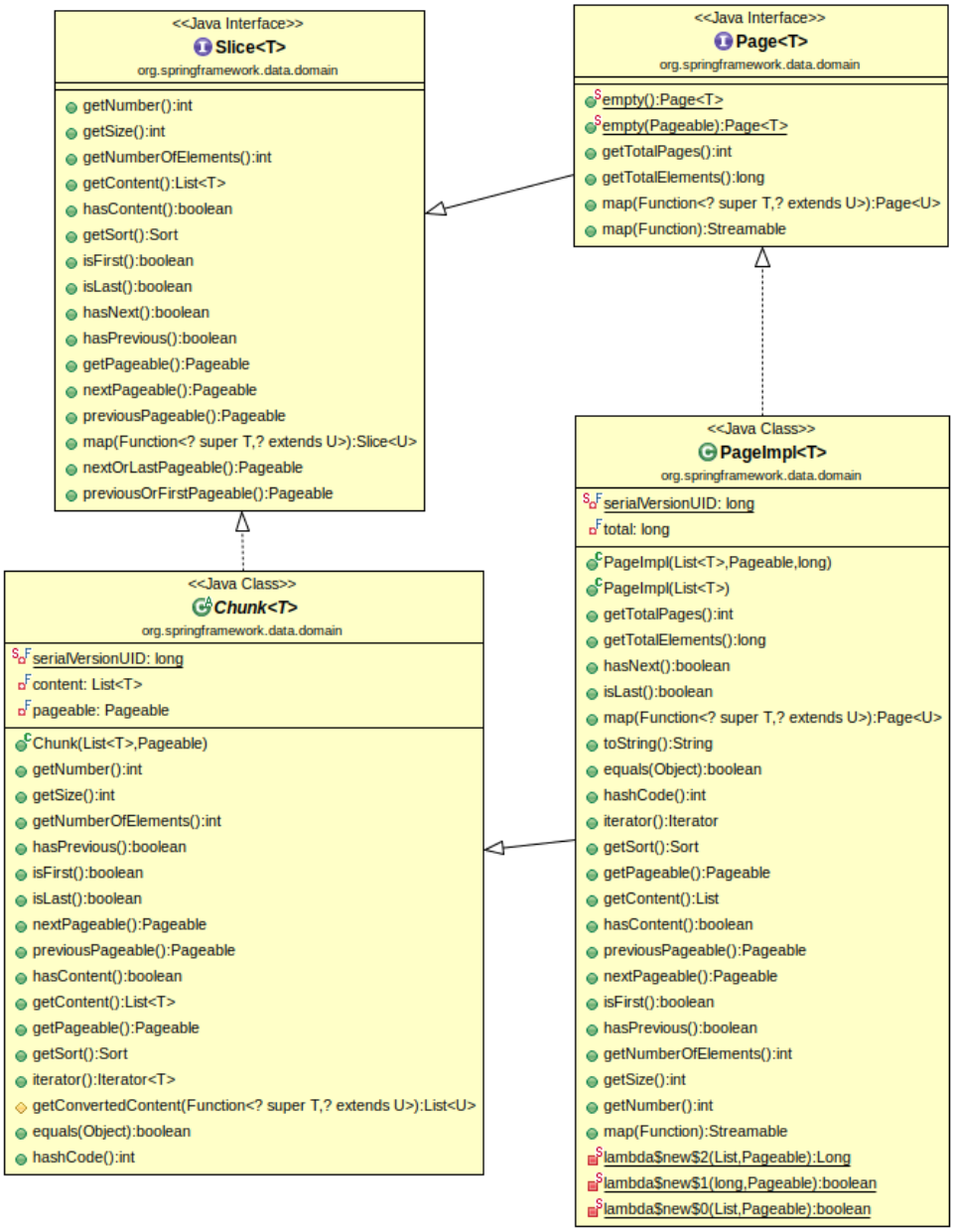
```
SELECT COUNT(p) FROM Persona p WHERE p.ciudad.id = :ciudadId
```

En consecuencia, cada búsqueda paginada requiere dos consultas: la que obtiene los resultados página a página y la que cuenta los resultados existentes.

1.2.4.13.2. La interfaz Page<T>

Spring Data cuenta con una herramienta específica que se encarga de la consulta de conteo, así como de utilizar el resultado para calcular el total de páginas. Su uso es tan sencillo como hacer que el método que representa a la consulta paginada devuelva la interfaz Page<T>.

A continuación podemos observar el diagrama de clases detallado:



Page<T> está tipada para el resultado «real» de la consulta: *T* es la clase de la entidad que corresponda. Para acceder a los resultados pertenecientes a la página, una lista de *T*, sólo es necesario invocar el método *getResult*.

La utilidad de **Page** radica en que contiene los detalles de la página devuelta por la consulta, incluyendo el número de la página actual y el total de ellas. Incluso provee los objetos *Pageable* y *Sort* empleados. Y todo ello sin necesidad de realizar ningún tipo de configuración adicional en el método del repositorio.

A continuación se muestran los métodos más interesantes de **Page**:

`long getTotalElements()`	El número total de resultados (el conteo).
`int getTotalPages()`	El número total de páginas.
`int getSize()`	El tamaño de la página.
`int getNumber()`	La página que representa el objeto `Page`.
`int getNumberOfElements()`	El número de resultados de la página. Si se trata de la última, no tienen por qué coincidir con el tamaño de la página (el método `getSize()`).
`boolean isFirst()`	Indica si la página es la primera.
`boolean isLast()`	Indica si la página es la última.
`boolean hasNext()`	Indica si hay más páginas.
`boolean hasPrevious()`	Indica si no es la primera página.
`Pageable getPageable()`	El `Pageable` usado.

A continuación podemos observar la notación de un método de búsqueda usando paginación con **Page** y un ejemplo de uso:

```
Page<Persona> findPersonaBy(Pageable pageable);
```

```
void pruebaPaginacion() {
    Pageable pag = PageRequest.of(0, PAGE_SIZE, Sort.Direction.ASC, "nombre");

    Page<Persona> primeraPagina = PersonaRepository.findPersonaBy(pag);

    System.out.println(primeraPagina.getContent());
    System.out.println(primeraPagina.getTotalElements());
    System.out.println(primeraPagina.getNumber());
    System.out.println(primeraPagina.getTotalPages());
    System.out.println(primeraPagina.getNumberOfElements());
    System.out.println(primeraPagina.getSize());
}
```

La clave para que todo funcione reside en la sentencia *SELECT COUNT* que Spring Data JPA genera y ejecuta para conocer todos los posibles resultados (el valor que devuelve *getTotalElements*).

En el caso de las consultas declaradas con *@Query* puedes indicar la consulta de conteo en la propiedad **countQuery** de la anotación:

```
@Query(value="SELECT p FROM Persona p",
       countQuery = "SELECT count(p) FROM Persona p")
```

```
Page<Persona> findPersonasJpql(Pageable pageable);
```

En todos los casos, sólo es necesario escribir la consulta de conteo cuando Spring Data JPA es incapaz de generarla de manera adecuada. En caso de no poder generarla automáticamente se lanzará una excepción al ejecutarse la consulta.